

DDBot: Differentiable Physics-Based Digging Robot for Unknown Granular Materials

Xintong Yang , Minglun Wei , *Student Member, IEEE*, Yu-Kun Lai , *Senior Member, IEEE*,
and Ze Ji , *Member, IEEE*

Abstract—Automating the manipulation of granular materials poses significant challenges due to complex contact dynamics, unpredictable material properties, and intricate system states. Existing approaches often fail to achieve efficiency and accuracy in such tasks. To fill the research gap, this article studies the small-scale and high-precision granular material digging task with unknown physical properties. A key scientific problem addressed is the feasibility of applying first-order gradient-based optimization to complex differentiable granular material simulation and overcoming associated numerical instability. A new framework, named differentiable digging robot (DDBot), is proposed to manipulate granular materials, including sand and soil. Specifically, we equip DDBot with a differentiable physics-based simulator, tailored for granular material manipulation, powered by GPU-accelerated parallel computing and automatic differentiation. DDBot can perform efficient differentiable system identification and high-precision digging skill optimization for unknown granular materials, which is enabled by a differentiable skill-to-action mapping, a task-oriented demonstration method, gradient clipping and line search-based gradient descent. Experimental results show that DDBot can efficiently (converge within 5 to 20 minutes) identify unknown granular material dynamics and optimize digging skills, with high-precision results in zero-shot real-world deployments, highlighting its practicality. Benchmark results against state-of-the-art baselines also confirm the robustness and efficiency of DDBot in such digging tasks.

Index Terms—Autonomous robots, computational modeling, deformable models, industrial robots, robots, robot control, robot learning, system identification, trajectory optimization.

I. INTRODUCTION

GRANULAR material is everywhere in our daily life, ranging from sand, soil, rocks, rice, sweetcorn, beans, etc. Automating the manipulation of such materials is of vital importance. For instance, it is highly valuable to the development of a robotic farm or greenhouse, where machines need to dig soil for planting. It is also of high value to robotic excavation

tasks, which play an important role in disaster rescue missions or object retrieval in out-of-reach places. Moreover, it is also applicable to soil sampling tasks that need to be carried out in dangerous caves, mountains, or even other planets that humans cannot reach. Unfortunately, state-of-the-art robotic manipulation methods still struggle to handle such materials with high precision, flexibility and efficiency.

In this work, we aim to develop a high-precision and efficient framework for granular material manipulation, designed for real-world tasks with unknown material properties, extended task horizons, and diverse manipulation targets. Our approach focuses on small-scale but high-precision digging tasks with varying target sizes and locations. Such tasks typically require a robot to precisely control the depth and location to insert a shovel and the angle and distance to push the material.

The first challenge in developing a practical granular material manipulation system lies in the accurate and efficient prediction of contact dynamics, which is crucial for optimizing robotic manipulation solutions [1]. Advances in simulation techniques have significantly progressed rigid object manipulation in the last few decades. In recent years, numerical modeling techniques, such as the material point method (MPM) and its variants, have greatly improved the fidelity and efficiency of simulations of nonrigid materials [2], [3]. MPM models nonrigid materials as particle collections, calculating dynamics based on physical laws on the particles and a background Eulerian grid, achieving superior accuracy compared to data-driven approaches [4]. However, it remains unintuitive to tune the values of granular material properties, such as Young's modulus, Poisson's ratio, density, and friction angle, to match the real-world dynamics, hindering the development of high-precision manipulation solutions. While differentiable physics-based system identification (DPSI) has demonstrated efficiency for deformable objects, like plasticine [5], its feasibility on granular materials remains unexplored.

Second, there is a lack of investigation of how state-of-the-art planning and control methods perform in such tasks. The most popular learning-based method for robotic manipulation in recent years is arguably deep reinforcement learning (DRL), which has proved successful in many robotic tasks [1]. DRL methods seek to learn a policy that outputs actions to maximize a certain reward function via (mostly) randomized exploration [6]. However, DRL is notoriously data-hungry, and can be easily stuck in local minima due to limited positive feedback. DRL

Received 27 July 2025; accepted 17 October 2025. Date of publication 27 November 2025; date of current version 11 December 2025. This work was supported by the U.K. Engineering and Physical Sciences Research Council (EPSRC) under Grant EP/X018962/1. The work of Minglun Wei was supported by EPSRC Doctoral Training Partnership under Grant EP/W524682/1. This article was recommended for publication by Associate Editor F. Abu-Dakka and Editor M. Schwager upon evaluation of the reviewers' comments. (*Corresponding author: Ze Ji.*)

Xintong Yang, Minglun Wei, and Ze Ji are with the School of Engineering, Cardiff University, CF24 3AA Cardiff, U.K. (e-mail: yangx66@cardiff.ac.uk; weim9@cardiff.ac.uk; jiz1@cardiff.ac.uk).

Yu-Kun Lai is with the School of Computer Science and Informatics, Cardiff University, CF24 4AG Cardiff, U.K. (e-mail: lai4@cardiff.ac.uk).

Digital Object Identifier 10.1109/TRO.2025.3636815

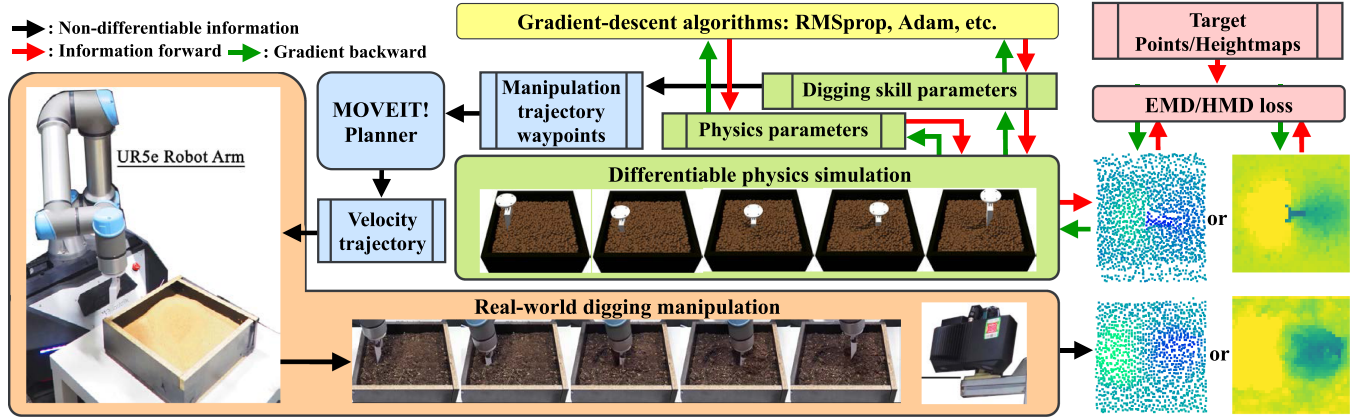


Fig. 1. DDBot (Differentiable Digging Robot) system overview. DDBot has five modules. 1) A robot platform (orange) that performs real-world manipulation and collects real-world surface point clouds of the granular materials. The five small images visualize a real-world digging trajectory that matches the trajectory optimized in simulation shown in the green box. 2) A differentiable physics simulator (green) that simulates the digging skill trajectory given the physics and skill parameters. 3) A differentiable loss function (pink) that compares the geometrical differences between the simulated and real manipulation results. 4) A gradient-based optimization algorithm (yellow) that updates the physics or skill parameters. 5) The Robot Operating System (ROS) communication protocol and the MoveIt! motion planning library [16] that generates a velocity trajectory from the skill parameters to control the real robot.

also struggles to generalize to unseen scenarios without fine-tuning or retraining. On the other hand, model predictive control (MPC) is another set of common methods for robotic planning and control [7]. MPC typically applies stochastic sampling or evolutionary methods to optimize a distribution of the manipulation motion with a specified cost function (or fitness function). Despite its successes in various domains, MPC methods tend to evolve solutions based on a cost function that evaluates the system's final state, lacking consideration of the physical dynamics. It ignores the connection between the manipulation skills and the resultant manipulation outcomes, leading to poor local minima and poor interpretability of the found solutions. In addition, it remains unclear how it may perform in such granular manipulation tasks.

Recently, an emerging set of robotic manipulation methods that rely on differentiable physics-based simulators has shown promising results in generating manipulation solutions for non-rigid materials [8], [9]. These methods combine the realism and accuracy of the laws of physics (e.g., MPM), and the automated differentiation mechanism and parallel computing technique of the simulator (e.g., Torch, TaiChi) [10], [11] to calculate physically meaningful gradients of the manipulation trajectory with respect to a loss function. Though researchers have demonstrated the feasibility of direct gradient-based trajectory optimization with nonrigid materials that are modeled by the fixed corotated elastic energy and the von Mises plastic criterion [3], [12], no work has been done to study the feasibility of gradient-based methods on granular materials, which are more accurately modelled by the St. Venant-Kirchhoff (SVK) elastic energy and the Drucker-Prager (DP) plastic yield criterion [13]. In addition, first-order gradient descent methods, despite their efficiency, are sensitive to numerical instability, particularly in large systems with long trajectories and rugged loss landscapes [14]. Thus, there is a need to study the feasibility of gradient-based optimization for granular material manipulation.

To fill the research gap, this article makes the following contributions to the robotization of real-world, small-scale and high-precision granular material digging tasks with unknown material properties.

First, to the best of our knowledge, this is the first attempt to develop an efficient and high-fidelity differentiable simulator specifically designed for granular material manipulation. It supports efficient forward dynamics computation to train data-driven methods, such as DRL, and evaluate nonlearning methods, such as MPC. It also enables efficient automatic differentiation to support gradient-based optimization methods, such as RMSProp [15].

Second, based on the proposed simulator, we present a novel framework, named differentiable digging robot (DDBot) (Fig. 1), which allows first-order gradient-based optimization in system identification tasks and small-scale high-precision digging tasks for real-world granular materials with unknown properties. We experimentally show that gradient-clipping is effective in stopping gradient explosion in SVK and DP-based granular dynamics. Video illustration is available on YouTube. Codes are publicly available on Github.

DDBot uses a differentiable digging skill to significantly reduce the solution space and improve manipulation quality, initializes manipulation solution before optimization via target-oriented demonstrations to accelerate convergence, and performs line search-based gradient descent with adaptive step sizes to stabilize convergence.

Last, we experimentally confirm the feasibility of differentiable system identification with soil and sand materials. For digging skill optimization, we evaluate and benchmark, for the first time, the performances on a series of soil and sand digging tasks with state-of-the-art manipulation optimization methods, confirming the robustness and efficiency of our DDBot system. Evaluation on a real-world UR5 robot arm is performed to confirm the practicality of the optimized digging motions.

The rest of this article is organized as follows. Section II discusses existing literature related to this work. Section III presents our method in detail. Section IV presents the experiment design and summarizes the key questions for investigation. Section V addresses the unstable gradients and rugged loss landscapes. Section VI discusses the results of system identification, followed by the results of digging skill optimization in Section VII. Section VIII presents the performance benchmark and ablation study. Finally, Section IX concludes this article.

II. RELATED WORK

A. Robotic Soil Manipulation Tasks

A vast majority of robotic digging research focuses on soil excavation tasks in the construction context, where a large amount of soil, sand, and rocks are to be removed [17], [18], [19], [20]. An early work presented an autonomous truck-loading system that searches for a starting location of the bucket of a given digging motion to fill the bucket with soils [17]. More recently, methods have been proposed to tackle such excavation tasks with uncertain soil properties and unpredictable soil-bucket interaction forces via hierarchical optimization [18] and reinforcement learning [19]. Another work also studied the sim-to-real gap for soil loading tasks and demonstrated the possibility of reaching low sim-to-real differences with the discrete element method (DEM) for soils with known and uniform physical properties [20].

In agriculture, heavy machines play a key role in large-scale ploughing, seed-sowing, and harvesting tasks but are typically equipped with tailored soil-digging tools and carried out in a large farming field by a human operator [21], while agricultural robots for planting or seed-sowing are mostly designed as large vehicles operating in large outdoor fields [22], [23]. In terms of indoor and small-scale soil-related manipulations, such as greenhouses and household gardening tasks, not much effort has been spent on automated soil manipulation with precision, flexibility, and adaptability.

Beyond agriculture, soil manipulation is also required to be automated for soil sampling tasks, either for space exploration [24] or determining farming field soil properties [25]. Though these operations are done in a much more constrained space, they are typically carried out by specialized tools, such as drills, that do not consider changing soil properties nor require flexible and high-precision control.

The disadvantages of these approaches are primarily twofold. First, they are restricted to a single manipulation motion, inapplicable to ever-changing task requirements and unknown soil properties. Second, they were developed for large-scale operations without the need to satisfy high-precision small-scale control, which is heavily challenged by the difficulty of precisely modeling the physical processes of the materials. Against this background, we develop a robotic granular material manipulation method that deals with *high-precision* small-scale operations in unstructured environments with *changing task requirements* (varying hole shapes) and *unknown material properties*.

B. Robotic Granular Material Manipulation Methods

After several decades, robotic manipulation methods have evolved from hand-crafted motions and simplified model-based motion planning [26] to data-driven (learning-based) approaches [1] and differentiable physics-based trajectory optimization [7], [11]. Our focus is on granular material manipulation, whose numbers of dimensions cause classic motion planning methods to struggle, and only a few works in recent years have dealt with such materials.

For granular materials, effort has been devoted to simplifying the representation and dynamics. The authors in [27] trained a reinforcement learning (RL) policy to shape granular materials on a table represented by at most 200 particles in a simulator powered by position-based dynamics (PBD). Due to the costs of high-fidelity simulations, The authors in [28] studied a granular material removal task by simplifying the dynamics into a 2-D shape control problem represented by a Gaussian mixture model and optimizing the control velocity in the task space to achieve a target shape. The authors in [29] employed a graph neural network to estimate the particle states of granular material being poured out from a cup and plan trajectories for different desired pouring outcomes. Their training data was, however, collected using Taichi and MPM-based simulation with about 1400 particles. The authors in [30] proposed to avoid the costly computation of the physics-based interactions by using a fully convolutional neural network to achieve differentiable rendering and thus gradient-based trajectory optimization for tabletop granular material gathering tasks. The authors in [31] proposed a metalearning approach to predict the scooped volume of different granular materials of a scoop action from RGB-D image patches and used Bayesian optimization to find the parameters of a simplified scoop motion that maximizes the scooped volume. Another recent work also presented a framework to train RL policies for granular material transportation tasks using demonstrations collected by gradient-based trajectory optimization in a differentiable fluid simulation environment to bypass gradient explosion [14].

In summary, there is no existing approach that targets *high-precision* digging tasks, which are challenged by the difficulty of simulating the high-fidelity dynamics of the target granular material with *unknown properties*. This article fills this gap by studying such tasks. Method-wise, existing approaches are inefficient with high-fidelity physics dynamics and have to trade off manipulation precision to increase computation efficiency. On the contrary, our novel gradient-based digging skill optimization method can maintain both high manipulation precision and computational efficiency through a GPU-accelerated parallel computing-supported simulator.

III. METHOD

This section presents details of our method for robotic soil/sand digging. An overview of the proposed DDBot system is given in Fig. 1. The primary novelties of the DDBot system are threefold: 1) the use of GPU-accelerated, parallel computing-supported, and MPM-based physics simulation for granular material dynamics that is both time-efficient and physically

precise; 2) the complete differentiability of the physics simulator that allows efficient gradient descent algorithms to optimize physics parameters and manipulation skills/trajectories; and 3) a task-oriented digging demonstration method that provides initial solutions to accelerate skill optimization. With the use of gradient-clipping and line search-based gradient update, DDBot can accomplish highly precise granular digging tasks in ~ 5 minutes. The rest of this section starts by mathematically formalizing the digging task and the proposed digging skill, introducing the skill demonstration, discussing the differentiability of the system components, and explaining how to stop gradient explosion and converge over rugged losses.

A. Problem Definition

We adopt the partially observable goal-conditioned Markov decision processes (POGCMDBs) to mathematically describe the soil/sand digging problem, in which a robot takes actions to interact with the environment to accomplish a task goal. Formally, the POGCMDBs comprise a set of observations $o \in \mathcal{O}$, a set of system states $s \in \mathcal{S}$, a set of goals $g \in \mathcal{G}$, a set of actions $a \in \mathcal{A}$, an initial state distribution $p(s_0)$, a (deterministic) system dynamics function $s' = f(s, a) : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$, a goal-conditioned reward function, i.e., a loss function $r(s, a, \hat{g})$, where $\hat{g} \in \hat{\mathcal{G}} \subset \mathcal{G}$ is a desired goal and $\hat{\cdot}$ denotes *desired* or *target*, and a discount factor γ .

States and observations: In our problem, we focus on a real-world digging environment, where a UR5e robot arm equipped with a 3D-printed shovel is tasked to dig holes in a container full of soil/sand. While we assume a system state $s \in \mathcal{S}$ that comprises all the soil/sand particles' positions, velocities, and accelerations in the world frame (in both reality and simulation), realistically the robot can only observe the surface point clouds (Cartesian points) of the soil/sand obtained from a 3D optical camera. In other words, $\mathcal{O} = \{x | x \in \mathcal{X}_{surface}\}$ where $\mathcal{X}_{surface} \subset \mathbb{R}^3$ is the set of observed surface points. For clearer reading, we overload o to denote both states and observations in the following text and use x to denote Cartesian coordinates.

In practice, the surface points in our experiments are generated by transforming the original point clouds from the camera frame to the world frame, cropped within the area of the container and downsampled with a 40×40 grid of size 0.24, centred at the centre of the soil/sand container. In the simulation, this observation is generated by selecting the highest particles in a 40×40 square grid of size 0.24 m, centred at the centre of the soil/sand container. Both observations contain 1600 particles.

The initial state distribution of our problem is defined as the soil/sand being flattened in the container at a certain height (0.07 meters). In the simulation, the particle body is created by filling a $0.28 \times 0.28 \times 0.07$ box with a fixed density (5×10^6 particles per m^3), which results in 27440 simulated particles. The radius of the particle is determined by dividing the box by the number of particles created ($r_p = 2 \times 10^{-7} \text{ m}^2$). Manually flattening the soil/sand is required to reset the real-world system to its initial state. The initial state of the robot is defined as the end-effector being positioned on top of the centre of the soil/sand container, with the tip of the shovel touching the surface of the soil/sand

and orientated to face its digging side towards the positive x direction.

Goal and reward/loss: For digging tasks, a meaningful dense reward function is difficult to define. Instead, desired goals are much easier to specify. We use the 3-D printed shovel to manually make the soil/sand into different shapes for different tasks (Fig. 4). The surface point clouds of the results of these manual operations are then taken as desired goals, i.e., $\hat{\mathcal{G}} = \{\hat{x} | \hat{x} \in \hat{\mathcal{X}}_{surface}\}$. The reward (loss) function is defined as the unnormalized Earth Mover's distance (EMD) between the next observation and a desired goal, irrelevant to the action

$$r_{EMD}(o', \hat{g}) = d_{EMD}(\mathcal{X}, \hat{\mathcal{X}}) = \sum_{x \in \mathcal{X}} \min_{\phi \in \phi^{EMD}: \mathcal{X} \rightarrow \hat{\mathcal{X}}} \|x - \phi(x)\|_2 \quad (1)$$

where ϕ^{EMD} denotes a one-to-one injective mapping that only exists when $|\mathcal{X}| \leq |\hat{\mathcal{X}}|$. In practice, a linear assignment algorithm is used to calculate ϕ^{EMD} .

In addition to the EMD distance, we also define a height map-based distance (HMD) function as an alternative reward/loss. In other words, the desired goal can also be represented by a target height map $\hat{\mathcal{I}}$. Given the resultant height map, $\mathcal{I}$, the height map-based reward is defined as the summed pixel difference

$$r_{HMD}(o', \hat{g}) = d_{HMD}(\mathcal{I}, \hat{\mathcal{I}}) = \sum_i \sum_j \|\mathcal{I}_{ij} - \hat{\mathcal{I}}_{ij}\|_2 \quad (2)$$

where i, j are pixel indexes. A height map is generated by traversing all points/particles, computing their referenced pixel indices from their x and y coordinates with an offset of the particle radius r_p . This links each particle to 5 pixel indices corresponding to $(x, y); (x + r_p, y); (x - r_p, y); (x, y + r_p); (x, y - r_p)$. The height map is initialized to zero, and the pixel values are replaced with z values of the particles when they are higher than the existing values. This avoids empty pixels in height map generation due to point sparsity.

Actions: An action of the robot manipulator at each step here is a 6D displacement vector in the Cartesian space (i.e., end-effector tip frame) $a = [\Delta x, \Delta y, \Delta z, \Delta a, \Delta b, \Delta c] \in \mathcal{A} \subset \mathbb{R}^6$, i.e., it controls the amount of translational changes in the x, y, z directions and the rotational changes about these three axes. In the real world, motion planning (MOVEIT! and RRT connect [16], [32], [33]) are used to compute a velocity trajectory from a series of Cartesian space waypoints (a trajectory of actions) to control the actual robot arm. In the simulation, the Cartesian displacement for each global step is divided evenly by the number of simulation substeps and applied to the pose of the end-effector before collision detection at each substep.

Parameterized skill: For long-horizon manipulation tasks, controlling the 6-D displacements in the Cartesian space is difficult due to the continuous high-dimensional solution space compounded over time. Specifically, a trajectory of T timesteps consists of $6T$ continuous variables, where T usually is greater than 300 in robot manipulation tasks with a simulation step size of 0.01s. Therefore, we employ temporal abstraction over the trajectory space by introducing a parameterized skill, denoted

as $\bar{a}(\theta) = \{a_0, a_1, \dots, a_T\}$, where $\theta \in [-1, 1]^{|\theta|}$ is a set of continuous parameters for calculating the action trajectory of the skill and the length T is determined by a skill-to-action mapping, $\bar{a}(\theta)$, which is task-specific (detailed in Section III-B). Although multiple skills can be defined, one skill is sufficient in this work. This leads to a one-step POGCMDP whose dynamics function can be written as $o' = f(o, \bar{a})$.

System dynamics: The dynamics function (or transition probability distribution in stochastic cases) determines the next state of the system given the current state and a given action. Obtaining a perfect function to model such real-world dynamics accurately is nearly impossible. However, it can be approximated using numerical modeling methods. In our work, we adopt the moving least squares material point methods (MLS-MPM) to simulate granular materials [4], coupled with the St. Venant-Kirchhoff (SVK) constitutive model for hyperelasticity and the Drucker-Prager (DP) yield criterion for sand/soil plasticity [13], [34].

MPM represents the soil/sand body as a collection of particles with positions, velocities, deformation gradients, and their material properties, including Young's modulus, Poisson's ratio, density, and sand friction angle. It efficiently integrates the system's velocity over time and handles collision and fracture on a background Eulerian grid [13]. MLS-MPM adopts a novel weak form discretization of the governing equations and replaces the shape functions in the force computation with MLS approximators, achieving faster and more realistic simulation and two-way coupling with rigid objects that classic MPM fails [4].

For the sake of clarity, we present a simplified procedure of MLS-MPM, the key equations of the constitutive models (hyperelasticity and plasticity) and a brief description of the collision contact process below. Please refer to [4], [5], [13] for detailed illustrations of the dynamic model.

As an explicit simulation algorithm, MLS-MPM advances simulation time in a global/local time-stepping process [4]. In other words, for the t th simulation step ($0 < t \leq T$) with a simulation stepsize Δt , there are N substeps with a sub-stepsize $\Delta t_{sup} = \Delta t/N$. Denote $\mathbf{x}_p^{n+1} = f_{sub}^{t,n}(\mathbf{x}_p^n, a_t, \Theta, \mathbf{g}, \Delta t_{sup})$ as the function of the n th substep of the t th step, it can be summarized as the list of consecutive functions shown in Algorithm 1.

In Algorithm 1, line 4, the constitutive model $f_{constitutive}$ first alters the deformation gradient F_p^{tmp} based on the plasticity model for any particle that undergoes plastic deformation. Then it computes the stress of that particle using the altered deformation gradient F'_p and the hyperelasticity energy model. With the singular value decomposition (line 3 of Algorithm 1) of the deformation gradient for particle p after the last iteration update, the SVK elastic energy density function can be written in terms of the diagonal matrix S_p

$$\psi_p(F) = \mu \text{tr}((\ln S_p)^2) + \frac{1}{2} \lambda (\text{tr}(\ln S_p))^2 \quad (3)$$

whose derivative is the first Piola–Kirchhoff stress tensor

$$\mathbf{P}_p = \frac{\partial \psi_p}{\partial F} = U_p (2\mu S_p^{-1} \ln S_p + \lambda \text{tr}(\ln S_p) S_p^{-1}) V_p^T \quad (4)$$

Algorithm 1: A Simplified MLS-MPM Substep Procedure ($f_{sub}^{t,n}$)

1. For each particle p :
 2. $F_p^{tmp} = (I + \Delta t_{sup} C_p) F_p$
 3. $U_p, S_p, V_p = f_{svd}(F_p^{tmp})$
 4. $F'_p, \sigma_p = f_{constitutive}(U_p, S_p, V_p, \Theta)$
 5. $\Delta \mathbf{v}_{grid} = f_{p2g}(\mathbf{x}_p, \mathbf{v}_p, \sigma_p, C_p, \Delta t_{sup})$
 6. $\mathbf{v}_{grid} += \Delta \mathbf{v}_{grid}$
 7. $\mathbf{x}'_{agent}, \mathbf{q}'_{agent} = f_{move_agent}(a_t, \Delta t_{sup})$
 8. $\mathbf{v}'_{grid} = \mathbf{v}_{grid} + \Delta t_{sup} \cdot \mathbf{g}$
 9. $\mathbf{v}'_{grid} = f_{collision}(\mathbf{x}'_{agent}, \mathbf{q}'_{agent}, \mathbf{x}'_{container}, \mathbf{v}'_{grid}, \Delta t_{sup})$
 10. For each particle p :
 11. $\mathbf{v}'_p, C'_p = f_{g2p}(\mathbf{v}'_{grid}, \Delta t_{sup})$
 12. $\mathbf{v}'_p = f_{collision}(\mathbf{x}'_{agent}, \mathbf{q}'_{agent}, \mathbf{x}'_{container}, \mathbf{x}_p, \mathbf{v}'_p, \Delta t_{sup})$
 13. $\mathbf{x}'_p = \mathbf{x}_p + \Delta t_{sup} \cdot \mathbf{v}'_p$
-

F_p : deformation gradient; C_p : affine matrix; U, S, V : results of singular value decomposition; Θ : Material parameters; σ_p : Cauchy stress; $'$ denotes next substep; a : agent action; $\mathbf{g}$: gravity vector; $\mathbf{x}, \mathbf{q}, \mathbf{v}$: position, rotation and velocity vectors.

where μ and λ are the Lamé constants and can be derived from the Young's modulus E and Poisson's ratio ν

$$\mu = \frac{E}{2(1+\nu)} \quad \lambda = \frac{E\nu}{(1+\nu)(1-2\nu)}. \quad (5)$$

Plasticity is most conveniently controlled by altering S_p . The DP yield criterion states the following. Let $\epsilon_p = \ln S'_p$, and

$$\hat{\epsilon}_p = \epsilon_p - \frac{\text{tr}(\epsilon_p)}{\dim} I_{dim} \quad \delta \gamma_p = \|\hat{\epsilon}_p\| + \frac{\lambda \dim + 2\mu}{2\mu} \text{tr}(\epsilon_p) \alpha_f \quad (6)$$

where $\alpha_f = \sqrt{\frac{2}{3} \frac{2 \sin \phi_f}{3 - \sin \phi_f}}$, $\dim$ is the spatial dimension of the problem (3 in our case), I_{dim} denotes the identity matrix of dimension $\dim$, $\delta \gamma_i$ is the amount of plastic deformation, and ϕ_f is the friction angle of sand/soil. The DP model alters S_p under different conditions as follows:

$$\hat{S}_p = \begin{cases} I_{dim} & \text{tr}(\epsilon_p) > 0, \\ S_p & \text{tr}(\epsilon_p) \leq 0 \text{ \& } \delta \gamma_p \leq 0, \\ \exp(\epsilon_p - \delta \gamma_p \frac{\hat{\epsilon}_p}{\|\hat{\epsilon}_p\|}) & \text{tr}(\epsilon_p) \leq 0 \text{ \& } \delta \gamma_p > 0. \end{cases} \quad (7)$$

Please refer to [13] for details of the three conditions. With $\hat{S}_p$, the deformation gradient after plasticity is $F'_p = U_p \hat{S}_p V_p^T$ and the Cauchy stress $\sigma_p = \frac{1}{\det(F'_p)} \mathbf{P}'_p F'^T_p$, where $\mathbf{P}'_p$ is computed by substituting $\hat{S}_p$ into (4), and $\det(F'_p) = 1$ is imposed to enforce incompressibility.

Given an action (Cartesian space displacement) at time t , the 6-D Cartesian pose of the manipulation (agent) is updated by evenly distributing the action over the N substeps and adding it to the manipulator pose at each substep (line 7 in Algorithm 1). The collision behaviors between the particles/grid and the manipulator are modelled by a signed distance field-based collision detection mechanism and the Columb frictional contact model, following the practices of previous works [4], [5].

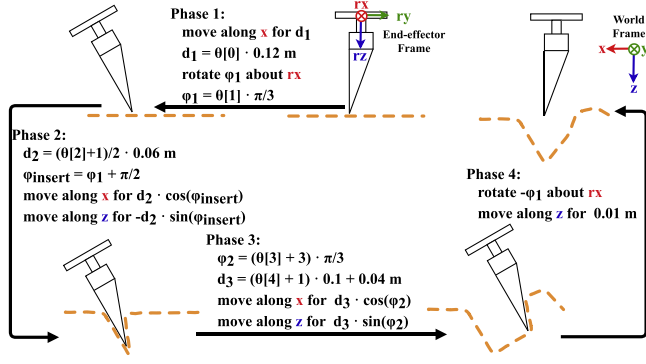


Fig. 2. Visualizing the digging skill modeled by (8) to (22). Orange dashed line: Soil or sand.

TABLE I
NOTATIONS USED IN THE DIGGING SKILL DEFINITION AND ITS DERIVATIVES ((8) TO (33))

Symbol	Meaning
$\theta_{displace}$	parameter: translational displacement in the world's x direction
θ_{rotate}	parameter: rotational displacement about rx (end-effector's x direction)
θ_{insert_dist}	parameter: insertion distance
θ_{push_dist}	parameter: pushing distance
θ_{push_ang}	parameter: pushing angle
d_i	translational displacement in phase i (m)
φ_i	rotational displacement in phase i (rad)
v_l	endeffector linear speed (m/s)
v_w	endeffector angular speed (rad/s)
Δt	simulation time stepsize
T_i^{float}	realvalued timesteps required in phase i
T_i	integer timesteps used in phase i
$max()$	returning the maximal value among the inputs
$round()$	returning the rounded value of an input
$exp()$	returning the exponential of the inputs
$det()$	returning the determinant of the inputs
$\Delta x_i, \Delta z_i$	perstep translations along x, z axes for phase i (m)
$\Delta r x_i$	perstep rotation about rx in phase i (rad)
$\bar{a}[i:j]$	the i -th (inclusive) to the j -th (exclusive) actions of the skill
$a_t[i]$	the i -th element of the t -th action of the skill

B. Digging Skill Design and Its Differentiability

This subsection presents the details of a parameterized digging skill. It produces a variable-length trajectory: $\bar{a}(\theta) = \{a_0, a_1, \dots, a_T\}$, where $\theta = \{\theta_{displace}, \theta_{rotate}, \theta_{insert_dist}, \theta_{push_angle}, \theta_{push_dist}\}$.

The skill comprises four phases graphically illustrated by Fig. 2 and mathematically defined by (8) to (22), with a list of notations comprised in Table I. Algorithm 2 presents the assembly of the calculated end-effector displacements into the complete trajectory $\bar{a}(\theta)$.

Phase 1: moving the shovel in the world's x axis by a distance d_1 and rotating about the shovel's x axis by an angle φ_1 . (8) to (11) define the calculation of the actions for phase 1 from $\theta_{displace}$ and θ_{rotate} .

$$d_1 = \theta_{displace} \cdot 0.12, \quad \varphi_1 = \theta_{rotate} \cdot \pi/3 \quad (8)$$

$$T_{d_1}^{float} = |d_1 / (v_l \cdot \Delta t)|, \quad T_{\varphi_1}^{float} = |\varphi_1 / (v_w \cdot \Delta t)| \quad (9)$$

$$T_1^{float} = \max(T_{d_1}^{float}, T_{\varphi_1}^{float}), \quad T_1 = \text{round}(T_1^{float}) \quad (10)$$

$$\Delta x_1 = \begin{cases} \frac{d_1}{T_1}, & T_1 > 0 \\ 0, & T_1 = 0 \end{cases}, \quad \Delta r x_1 = \begin{cases} \frac{\varphi_1}{T_1}, & T_1 > 0 \\ 0, & T_1 = 0 \end{cases} \quad (11)$$

Phase 2: inserting the shovel into the soil/sand by a distance d_2 along its pointing direction. (12) to (15) define the calculation of the actions for phase 2 from θ_{rotate} and θ_{insert_dist}

$$d_2 = (\theta_{insert_dist} + 1) \cdot 0.03, \quad \varphi_2 = \varphi_1 + \pi/2 \quad (12)$$

$$T_2^{float} = d_2 / (v_l \cdot \Delta t) \quad (13)$$

$$T_2 = \text{round}(T_2^{float}) \quad (14)$$

$$\Delta x_2 = \begin{cases} \frac{d_2 \cdot \cos(\varphi_2)}{T_2}, & T_2 > 0 \\ 0, & T_2 = 0 \end{cases}, \quad \Delta z_2 = \begin{cases} \frac{-d_2 \cdot \sin(\varphi_2)}{T_2}, & T_2 > 0 \\ 0, & T_2 = 0 \end{cases} \quad (15)$$

Phase 3: pushing towards an angle φ_3 by a distance d_3 . (16) to (19) define the calculation of the actions for phase 3 from θ_{push_dist} and θ_{push_angle}

$$d_3 = (\theta_{push_dist} + 1) \cdot 0.1 + 0.04, \quad \varphi_3 = (\theta_{push_angle} + 3) \cdot \pi/3 \quad (16)$$

$$T_3^{float} = |d_3 / (v_l \cdot \Delta t)| \quad (17)$$

$$T_3 = \text{round}(T_3^{float}) \quad (18)$$

$$\Delta x_3 = \begin{cases} \frac{d_3 \cdot \cos(\varphi_3)}{T_3}, & T_3 > 0 \\ 0, & T_3 = 0 \end{cases}, \quad \Delta z_3 = \begin{cases} \frac{d_3 \cdot \sin(\varphi_3)}{T_3}, & T_3 > 0 \\ 0, & T_3 = 0 \end{cases} \quad (19)$$

Phase 4: rotating back to straight and lifting for a constant distance $d_4 = 0.01$ m. (20) to (22) define the calculation of actions for phase 4

$$T_{d_4}^{float} = d_4 / (v_l \cdot \Delta t) \quad (20)$$

$$T_4^{float} = \max(T_{\varphi_1}^{float}, T_{lift}^{float}), \quad T_4 = \text{round}(T_4^{float}) \quad (21)$$

$$\Delta r x_4 = \frac{-\varphi_1}{T_4}, \quad \Delta z_4 = \frac{d_4}{T_4} \quad (22)$$

For each phase, the number of timesteps is calculated by dividing the total displacements over a given linear or angular velocity times Δt and rounding it into an integer. As the rounding operation ((10), (14), (18) and (21)) is discontinuous and thus not differentiable, using it to calculate the actions will lead to incomplete gradients. In other words, the changes in actions can only backpropagate through the computation of the 6-D displacements for each action, but not through the computation of the number of timesteps (e.g., in phase 1, $\frac{\partial \Delta x_1}{\partial d_1} \neq 0$ while $\frac{\partial \Delta x_1}{\partial T_1} = 0$).

Algorithm 2: Skill-to-Action Mapping $\bar{a}(\theta) = \{a_0, \dots, a_T\}$.**Inputs:** $\theta_{displace}, \theta_{rotate}, \theta_{insert_dist}, \theta_{push_dist}, \theta_{push_angle}$
 $d_{lift}, v_l, v_w, \Delta t$.**Output:** $\bar{a} = \{a_0, a_1, \dots, a_T\}$

With (8) to (22), Compute:

 $T_1, T_2, T_3, T_4, \Delta x_1, \Delta r x_1, \Delta x_2, \Delta z_2, \Delta x_3, \Delta z_3, \Delta r x_4,$
 Δz_4 Let $T = T_1 + T_2 + T_3 + T_4$ For t from 0 to T : $a_t = [0.0, 0.0, 0.0, 0.0, 0.0, 0.0]$ For t from 0 to $T_1 - 1$: $a_t[0] = \Delta x_1, a_t[3] = \Delta r x_1$ For t from T_1 to $T_2 - 1$: $a_t[0] = \Delta x_2, a_t[2] = \Delta z_2$ For t from T_2 to $T_3 - 1$: $a_t[0] = \Delta x_3, a_t[2] = \Delta z_3$ For t from T_3 to $T_4 - 1$: $a_t[3] = \Delta r x_4, a_t[2] = \Delta z_4$

An alternative is to assign the actions by dividing the total displacement over the pre-rounded number of simulation steps (replacing T_i with T_i^{float}). However, in phase 2, Δx_2 and Δz_2 become only related to φ_2 with this alternative (i.e., $\Delta x_2 = \frac{d_2 \cdot \cos(\varphi_2)}{T_2^{float}} = v_l \cdot \Delta t \cdot \cos(\varphi_2)$), which renders the gradient of θ_{insert_dist} effectively zero. In phase 3, this also leads to zero gradients for θ_{push_dist} . To avoid this, T_2 and T_3 (the integer versions) are retained for this alternative version of the skill-to-action mapping.

We present a complete derivation of the gradient chains for $\frac{\partial \bar{a}}{\partial \theta}$ in Section S.I in the Supplementary Materials. Ablation experiments are conducted to examine the effect of using an unrounded number of timesteps for per-action displacement calculation in phases 1 and 4. The derived gradients of θ with rounded numbers of timesteps are as follows:

$$\frac{\partial \bar{a}}{\partial \theta_{displace}} \Big|_{rounded_T_1} = \frac{\partial \bar{a}[0 : T_1]}{\partial \theta_{displace}} \Big|_{rounded_T_1} = 0.12 \quad (23)$$

$$\begin{aligned} \frac{\partial \bar{a}}{\partial \theta_{rotate}} \Big|_{rounded_T_1 T_2 T_4} &= \frac{\partial \bar{a}[0 : T_1]}{\partial \theta_{rotate}} \Big|_{rounded_T_1} + \frac{\partial \bar{a}[T_1 : T_1 + T_2]}{\partial \theta_{rotate}} \Big|_{rounded_T_2} \\ &+ \frac{\partial \bar{a}[T_1 + T_2 + T_3 : T_1 + T_2 + T_3 + T_4]}{\partial \theta_{rotate}} \Big|_{rounded_T_4} \\ &= \frac{\pi}{3} - d_2 \cdot \frac{\pi}{3} \cdot \left(\sin(\varphi_1 + \frac{\pi}{2}) + \cos(\varphi_1 + \frac{\pi}{2}) \right) - \frac{\pi}{3} \end{aligned} \quad (24)$$

$$\begin{aligned} \frac{\partial \bar{a}}{\partial \theta_{insert_dist}} \Big|_{rounded_T_2} &= \frac{\partial \bar{a}[T_1 : T_1 + T_2]}{\partial \theta_{insert_dist}} \Big|_{rounded_T_2} \\ &= 0.03 \cdot (\cos \varphi_2 - \sin \varphi_2) \end{aligned} \quad (25)$$

$$\begin{aligned} \frac{\partial \bar{a}}{\partial \theta_{push_dist}} \Big|_{rounded_T_3} &= \frac{\partial \bar{a}[T_1 + T_2 : T_1 + T_2 + T_3]}{\partial \theta_{push_dist}} \Big|_{rounded_T_3} \\ &= 0.1 \cdot [\cos(\varphi_3) + \sin(\varphi_3)] \end{aligned} \quad (26)$$

$$\begin{aligned} \frac{\partial \bar{a}}{\partial \theta_{push_angle}} \Big|_{rounded_T_3} &= \frac{\partial \bar{a}[T_1 + T_2 : T_1 + T_2 + T_3]}{\partial \theta_{push_angle}} \Big|_{rounded_T_3} \\ &= \frac{\pi \cdot d_3}{3} [\cos(\varphi_3) - \sin(\varphi_3)]. \end{aligned} \quad (27)$$

$$= \frac{\pi \cdot d_3}{3} [\cos(\varphi_3) - \sin(\varphi_3)]. \quad (28)$$

If unrounded numbers of timesteps are used (i.e., replacing T_1 and T_4), the gradients of $\theta_{displace}$ and θ_{rotate} become

$$\begin{aligned} \frac{\partial \bar{a}}{\partial \theta_{displace}} \Big|_{unrounded_T_1} &= \frac{\partial \bar{a}[0 : T_1]}{\partial \theta_{displace}} \Big|_{unrounded_T_1} \\ &= \begin{cases} T_1 \cdot \frac{-\pi \cdot \theta_{rotate} \cdot v_l \cdot \Delta t}{0.36 \cdot \theta_{displace}^2}, & T_{d_1}^{float} \geq T_{\varphi_1}^{float} > 0, \theta_{displace} > 0 \\ T_1 \cdot \left(\frac{2 \cdot v_l \cdot \Delta t}{\theta_{displace}} + \frac{\pi \cdot \theta_{rotate} \cdot v_l \cdot \Delta t}{0.36 \cdot \theta_{displace}^2} \right), & T_{d_1}^{float} \geq T_{\varphi_1}^{float} > 0, \theta_{displace} < 0 \\ T_1 \cdot \frac{0.12 \cdot v_w \cdot \Delta t}{|\theta_{rotate}| \cdot \pi/3}, & 0 < T_{d_1}^{float} < T_{\varphi_1}^{float} \end{cases} \end{aligned} \quad (29)$$

$$\begin{aligned} \frac{\partial \bar{a}}{\partial \theta_{rotate}} \Big|_{unrounded_T_1 T_2 T_4} &= \frac{\partial \bar{a}[0 : T_1]}{\partial \theta_{rotate}} \Big|_{unrounded_T_1} + \frac{\partial \bar{a}[T_1 : T_1 + T_2]}{\partial \theta_{rotate}} \Big|_{rounded_T_2} \\ &+ \frac{\partial \bar{a}[T_1 + T_2 + T_3 : T_1 + T_2 + T_3 + T_4]}{\partial \theta_{rotate}} \Big|_{unrounded_T_4} \end{aligned} \quad (30)$$

where

$$\begin{aligned} \frac{\partial \bar{a}[T_1 + T_2 + T_3 : T_1 + T_2 + T_3 + T_4]}{\partial \theta_{rotate}} \Big|_{unrounded_T_4} &= \begin{cases} T_4 \cdot \frac{-3 \cdot d_4 \cdot v_w \cdot \Delta t}{\theta_{rotate}^2 \cdot \pi}, & T_{\varphi_1}^{float} \geq T_{lift}^{float} > 0, \theta_{rotate} > 0 \\ T_4 \cdot \left(\frac{-2 \cdot v_w \cdot \Delta t}{\theta_{rotate}} + \frac{3 \cdot d_4 \cdot v_w \cdot \Delta t}{\theta_{rotate}^2 \cdot \pi} \right), & T_{\varphi_1}^{float} \geq T_{lift}^{float} > 0, \theta_{rotate} < 0 \\ T_4 \cdot \frac{-\pi \cdot v_l \cdot \Delta t}{3 \cdot d_4}, & 0 < T_{\varphi_1}^{float} < T_{lift}^{float} \end{cases} \end{aligned} \quad (31)$$

$$\begin{aligned} \frac{\partial \bar{a}[0 : T_1]}{\partial \theta_{rotate}} \Big|_{unrounded_T_1} &= \begin{cases} T_1 \cdot \frac{-0.36 \cdot \theta_{displace} \cdot v_w \cdot \Delta t}{\theta_{rotate}^2 \cdot \pi}, & T_{\varphi_1}^{float} > T_{d_1}^{float} > 0, \theta_{rotate} > 0 \\ T_1 \cdot \left(\frac{2 \cdot v_w \cdot \Delta t}{\theta_{rotate}} + \frac{0.36 \cdot \theta_{displace} \cdot v_w \cdot \Delta t}{\theta_{rotate}^2 \cdot \pi} \right), & T_{\varphi_1}^{float} > T_{d_1}^{float} > 0, \theta_{rotate} < 0 \\ T_1 \cdot \frac{\frac{\pi}{3} \cdot v_l \cdot \Delta t}{|\theta_{displace}| \cdot 0.12}, & 0 < T_{\varphi_1}^{float} \leq T_{d_1}^{float} \end{cases} \end{aligned} \quad (32)$$

$$\left. \frac{\partial \bar{a}[T_1 : T_1 + T_2]}{\partial \theta_{\text{rotate}}} \right|_{\text{rounded}_{T_2}} = -d_2 \cdot \frac{\pi}{3} \cdot \left(\sin(\varphi_1 + \frac{\pi}{2}) + \cos(\varphi_1 + \frac{\pi}{2}) \right). \quad (33)$$

C. Demonstration and Skill Prior

To improve the convergence speed, we rely on human priors to compose demonstrations to serve as initial guesses for skill optimization or to guide data collection for learning-based methods. We take inspiration from the fact that human children learn to manipulate by extrapolating a known skill over different manipulation scenarios and targets. In other words, given a set of skill parameters θ^- , we set θ^- as the initial skill parameters before running optimizations. The first parameter of the demonstration skill, θ_{displace} , is calculated based on the location of the hole presented in different task target point clouds

$$\theta_{\text{displace}} = \text{clip} \left(\frac{(\hat{\mathcal{X}}[\arg \min_z \hat{\mathcal{X}}][0] - 0.02)}{0.12}, -1.0, 1.0 \right)$$

where $\hat{\mathcal{X}}$ is the target surface point cloud. The rest of the demonstration skill parameters are constant: $\theta_{\text{rotate}} = 0.2$, $\theta_{\text{insert_distance}} = 0.8$, $\theta_{\text{push_angle}} = 0.0$, $\theta_{\text{push_distance}} = -0.5$, which gives a digging trajectory that digs a small hole near the hole presented in the task target.

D. Gradient-Based Skill Optimization

In classic trajectory optimization, the objective is to find a trajectory of T actions: $\tau = \{a_0, a_1, \dots, a_T\}$, such that the return is maximized

$$\tau^* = \arg \max_{\tau} r(o', \hat{g})|_{o' \sim f(o, \tau)}$$

where r is the reward/cost function, o' is the observation of the post-manipulation granular material, and $f(o, a)$ is the system dynamics. Instead of explicit optimization for every action in a fixed-length trajectory, we seek to optimize the proposed digging skill that produces a variable-length trajectory with five parameters, resulting in a much smaller solution space for optimization. The skill optimization problem with parameters θ can be formulated as

$$\begin{aligned} \max_{\theta} \quad & J(\theta) = r(o', \hat{g}) \\ \text{s.t.} \quad & o' = f(o, \tau) \\ & \tau = \bar{a}(\theta) \\ & \theta \in [-1, 1]^{|\theta|} \end{aligned}$$

where $J(\theta)$ is the objective function, $\bar{a}(\theta)$ is the skill-to-action mapping and θ is the set of skill parameters.

Many methods have been developed for such constrained optimization problems, yet gradient-descent methods are the most attractive due to faster convergence under the condition of the whole system being differentiable (smooth) [35]. We are seeking to compute $\partial r / \partial \theta$, and apply first-order gradient-descent algorithms to iteratively update θ towards local minima.

Three key components need to be differentiable (smooth) in computing the gradients of the skill parameters via the chain rule, i.e., the loss (reward) function r , the skill-to-action mapping $\bar{a}(\theta)$ and the dynamics function $f(o, \bar{a})$

$$\frac{\partial r}{\partial \theta} = \frac{\partial r(o', \hat{g})}{\partial o'|_{o'=f(o, \bar{a})}} \frac{\partial f(o, \bar{a})}{\partial \bar{a}} \frac{\partial \bar{a}}{\partial \theta}. \quad (34)$$

In this work, the loss function is either the earth mover's distance (1) or the height-map-based distance (2), which is either the sum of Euclidean distances of pairs of 3-D points or the sum of the differences of their z values, whose first-order derivatives (the first term of the right-hand side of (34)) are well-defined. The differentiability of the skill-to-action mapping (the third term of the right-hand side of (34)) was introduced in Section III-B with a full derivation given in Section S.I in the Supplementary Materials.

The differentiability of the dynamics: Here, we discuss the second term of the right-hand side of (34). As introduced in Section III-A, the dynamics function for soil/sand-rigid-body interactions is simulated by the MLS-MPM method [4]. The gradient chain of the t th simulation step comprising N substeps from the resultant observation o_{t+1} to the action a_t can be written as follows:

$$\begin{aligned} \frac{\partial o_{t+1}}{\partial a_t} &= \sum_{\mathbf{x}_p \in o_{t+1}} \sum_{n=0}^N \frac{df_{\text{sub}}^{t,n}(\mathbf{x}_p^{n-1}, a_t, \Theta, \mathbf{g}, \Delta t_{\text{sub}})}{da_t} \\ &= \sum_{\mathbf{x}_p \in o_{t+1}} \sum_{n=0}^N \left[\frac{\partial f_{\text{sub}}^{t,n}}{\partial \mathbf{x}_p^{n-1}} \cdot \frac{d\mathbf{x}_p^{n-1}}{da_t} + \frac{\partial f_{\text{sub}}^{t,n}}{\partial a_t} \right]. \end{aligned} \quad (35)$$

Algorithm 1 indicates that, as long as the collision function $f_{\text{collision}}$, the grid-to-particle function f_{g2p} , the agent position updating function $f_{\text{move_agent}}$, the particle-to-grid function f_{p2g} are differentiable over the particle positions and the agent's actions, one can obtain $\partial o_{t+1} / \partial a_t$ with (35) using the chain rule. The derivatives of these functions have been provided in [36]. Notice that the constitutive model, $f_{\text{constitutive}}$, needs not to be differentiable if only the gradients of actions are of interest. However, our work also seeks to perform system identification that optimizes the physics parameters for simulating soil/sand with gradient-based optimization (see Section IV-B). Therefore, we further provide proof of the differentiability of the constitutive model in Section S.II in the Supplementary Materials.

Combined with the discussion on differentiable skill in Section III-B, we establish the theoretical foundation for gradient-based digging skill optimization. The next two subsections will discuss the issues and solutions for exploding gradients and rugged loss landscapes that stop gradient descent from converging.

E. Stopping Gradients From Exploding

A critical issue that destabilizes or even disables gradient-based optimization with differentiable physics is the numerical instability of gradient computation caused by long task horizons and the large particle number. As mentioned in existing works, gradients tend to explode or vanish as the length of the computation chain (number of actions/simulation steps) or the number

of particles increases [2], [8]. This effect can be predicted based on (35), where $\partial o_{t+1}/\partial a_t$ depends heavily on the number of simulation steps T , substeps N , and particles $|o_{t+1}|$. It increases drastically (explodes) when there are too many sub-gradients that are greater than 1 ($\partial f_{sub}/\partial \mathbf{x}_p > 1$), and vice versa (vanishes). This problem is exacerbated when the physical model becomes more complex, preventing previous works from using efficient first-order optimization methods on tasks with long horizons or large numbers of particles [2], [3], [8], [9].

We experiment on three techniques to regularize the scale of the gradient for key variables in the MLS-MPM algorithm, namely *clipping*, *dynamic-scaling*, and *normalization*.

Specifically, denote a variable vector of interest as $\mathbf{x}$. The *clipping* operation bounds its gradient in $\nabla \mathbf{x} \in [-\nabla_{clip}, \nabla_{clip}]$ where ∇_{clip} is a user-defined threshold value.

The *dynamic-scaling* operation reduces the order of magnitude of the whole gradient vector by an integer such that the largest element of the gradient vector is kept below a certain order of magnitude: $\nabla \mathbf{x} \leftarrow \nabla \mathbf{x} / (10^{\Delta_{oom}} + \delta)$ where $\Delta_{oom} = \text{round}(\log(\max(\nabla \mathbf{x})) / \log 10 - oom^*)$ and oom^* is the maximal order of magnitude for the gradient defined by the user, and δ is a small value that prevents dividing by zero.

The *normalization* operation is straightforward, which divides a gradient vector by its magnitude (norm), $\nabla \mathbf{x} \leftarrow \nabla \mathbf{x} / (||\nabla \mathbf{x}|| + \delta)$, where δ is a small value. In our experiments, $\nabla_{clip} = 1e4$, $oom^* = 4$ and $\delta = 10^{-6}$.

Except for the material parameters in the SVK and DP models (Young's modulus E , Poisson's ratio ν , material density ρ , and sand friction angle ϕ_f) and the actions (skill parameters), the key variables in the MLS-MPM algorithm that play important roles in how the material behaves in response to external forces (actions) include the deformation gradient (F_p), position ($\mathbf{x}_p$) and velocity ($\mathbf{v}_p$) for each particle p and the grid mass m_{grid} and the velocities before and after collision detection $\mathbf{v}_{grid}$, $\mathbf{v}'_{grid}$. In Section V, we examine the vanilla gradient values of these variables and how the proposed techniques confine their scales.

F. Handling Rugged Loss and Fluctuating Gradients

Another cause that hinders efficient gradient descent is a rugged loss landscape, which causes gradients to fluctuate. Such fluctuating gradients lead to the divergence of the optimization process because they cause the so-called “overshooting” issue, a phenomenon in gradient-based optimization processes where the solution is updated with a stepsize that is too large such that it updates the solution too far in the gradient direction and passes over the local minimum, which happens very often with rugged loss landscapes. Unfortunately, because of the use of surface point clouds as the geometric representation of the materials, both the EMD and HMD losses present different levels of ruggedness at different sampling resolutions over the spaces of physics parameters and skill parameters. Typically, without any treatment, we observed that optimization with a fixed stepsize may encounter a good local minimum but cannot converge to that solution. For example, the red and pink curves in Fig. 6 show that vanilla optimization may reach a validation loss similar to optimizations with a line search technique (see

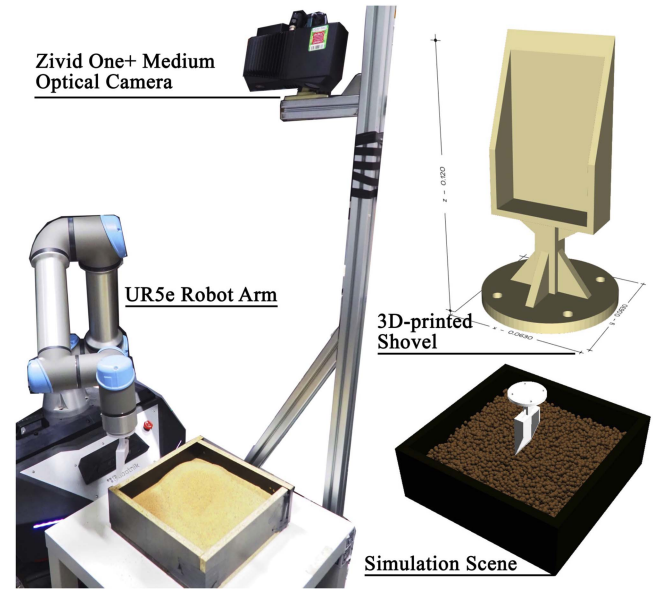


Fig. 3. Real manipulation platform.

below), but is unstable and exhibits high variance. A detailed examination of the loss landscapes and their ruggedness is given in Section V.

To address this challenge, we propose to use a simple yet effective technique called *line search*. Specifically, before each gradient update, we evaluate solutions that are updated by 5 different stepsizes along the gradient direction (by multiplying the stepsizes given in Section IV with $\alpha_{ls} \in \{0.1, 0.5, 1.0, 1.5, 2.0\}$). The one that minimizes the HMD loss is selected to update the solution, after which the optimization proceeds to the next iteration. The HMD loss is selected for line search evaluation because of its smoothness.

IV. EXPERIMENT DESIGN

A. Real-World Platform

We set up a UR5e robotic arm to perform digging manipulations in the real world. As shown in Fig. 3, the robot is equipped with a 3-D printed shovel as the end-effector. We place a $0.28 \times 0.28 \text{ m}^2$ wooden box with 0.11 metre depth to contain sand or soil for manipulation next to the robot. In the simulation, we use a box of the same size and the 3D model of the shovel as the simulated end-effector. We fill the container with the material, which could be sand or solid, to a height of 0.07 metres. A Zivid Medium One+ high-resolution camera [37] is used to collect real-world surface point clouds.

B. Soil/Sand System Identification

Before running skill learning and parameter optimization with the simulation dynamics, an important task is to ensure that the simulated soil/sand dynamics match the real ones as closely as possible. To achieve this, we follow the Differentiable Physics-based System Identification (DPSI) framework introduced in work [5], which can identify physics parameters with only one

TABLE II
STEPSIZES AND THE RANGES OF VALUES FOR THE PHYSICS PARAMETERS:
YOUNG'S MODULUS E , POISSON'S RATIO ν , MATERIAL DENSITY ρ , AND
SAND FRICTION ANGLE ϕ_f , SELECTED BASED ON AN OPEN-SOURCE
STRUCTURAL ENGINEERING DATASET [38]

Parameter	E (kPa)	ν	ρ (kg/m ³)	ϕ_f (deg)
Step size	10000	0.01	50	1
Min value	50000	0.1	1200	10
Max value	200000	0.4	2200	40

datapoint. In simple terms, the DPSI framework adjusts the key parameters of a physics model using gradients backpropagated from a loss function that compares the simulated and real point clouds of the same manipulation motion.

In this work, we seek to identify Young's modulus, Poisson's ratio, material density, and the friction angle of sand/soil for a more realistic simulation. MOVEIT! is used in this work to compute two manipulation trajectories—one for optimization and one for validation—for the system identification tasks.

The optimization trajectory moves the shovel from its initial configuration by +0.09 m in the x direction, -0.05 m in the z direction that inserts the shovel, -0.12 m in the x direction that pushes the material and another +0.12 m in the z direction that lifts the shovel. The validation trajectory moves the shovel to the corner of the container facing towards the diagonal corner, inserts it into the material for -0.05 m in the z direction, pushes the material diagonally (+0.12 m in both x and y directions), and lifts the shovel for +0.12 m in the z direction. For both soil and sand, the camera records the surface point clouds after the two trajectories as real-world ground truths. The velocity-based trajectories generated by MOVEIT! for system identification are transformed into a series of position-based waypoints with even timesteps for recreating the same end-effector motion in simulation.

The ranges of the physics parameters are defined according to an open-source structural engineering dataset [38]. They are summarized together with the gradient descent step sizes in Table II. The RMSprop algorithm with $\beta = 0.9$ [15] is used for gradient-based optimization in this work.

The feasibility of DPSI for physics parameter identification has been demonstrated with playdough-like plastic objects, which are governed by the fixed-corotated elastic energy model and the von Mises plasticity model [5]. In this article, we cope with granular materials that are modelled by the St. Venant-Kirchoff (SVK) elastic energy and the Drucker-Prager (DP) plastic yield criterion [13].

C. Digging Tasks

After the simulation is aligned with the real materials, we examine the feasibility of gradient-based skill optimization. We manually use the shovel to create 3 digging target shapes with both soil and sand. The optical camera then captures the resultant surface point clouds of the materials as task goals (see Fig. 4). We compare our skill optimization method to direct trajectory optimization (controlling 6-D Cartesian displacement at every global step), Covariance Matrix Adaptation MAP-Annealing

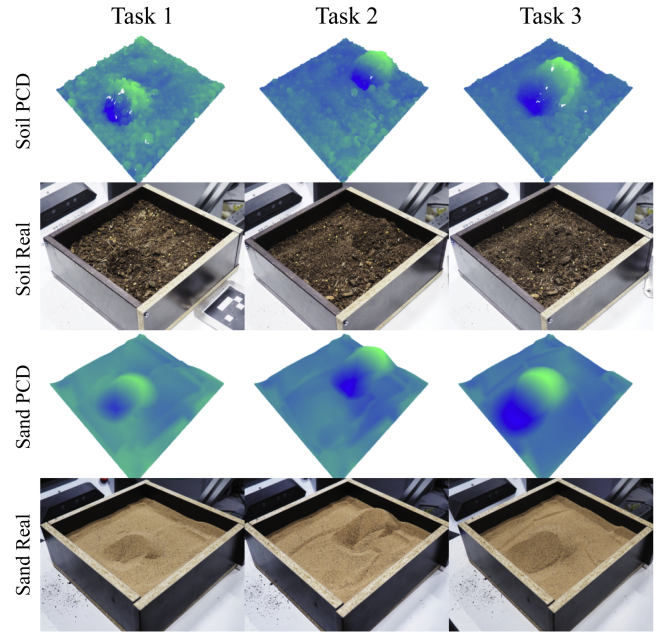


Fig. 4. Digging task goal visualization. PCD: point cloud.

(CMA-MAE) [39], [40] and goal-conditioned SAC (see Section S.III in the Supplementary Materials for implementation details).

For both gradient-based skill and trajectory optimization, we perform 20 gradient updates. RMSprop is used for gradient-based optimization with a stepsize of 0.03 for skill parameters and 0.004 for trajectory optimization, respectively.

CMA-MAE is a state-of-the-art evolutionary algorithm that does not use gradients to optimize a solution but optimizes the problem by searching for a batch of high-quality and diverse solutions [39], [40]. For each task, it is run with two optimization directions (i.e., two emitters) with a solution batch size of 10 for 20 iterations with a stepsize of 0.03.

The SAC agent is trained for 200 episodes, which results in 200 datapoints. Exploration is performed for 50 episodes by sampling the parameters from a Gaussian exploration strategy with the demonstration skill parameters as the mean. The critic and policy are updated 10 times with a batch size of 10 at every episode with a learning rate of 0.001. At every 5 training episodes, the policy is evaluated for 5 episodes to record its performance over training.

D. Experiment Questions

We present our experiment in four parts: 1) gradient and loss examination; 2) differentiable system identification; 3) differentiable skill optimization; and 4) benchmarking and ablation.

In Section V, we take a closer look at the behaviors of the gradients and the ruggedness of the loss landscapes. In Section VI, we examine the performance of system identification tasks on soil and sand with the proposed gradient regularization and step size treatments. In

Section VII, we examine the performance of the gradient-based skill optimization methods with and without skill demonstrations using the most performant gradient treatment found in the system identification experiments. In Section VIII, we benchmark the performances of the baselines and examine the gradient-based optimization performance with and without rounding the number of timesteps in the differentiable digging skill.

V. RESULTS—VISUALIZING GRADIENTS AND LOSSES

Exploding gradients: In Section S.IV in the Supplementary Materials, we visualize how the gradients of the key MLS-MPM variables, physics parameters, and actions evolve over the simulation's backwards direction. It clearly shows the effectiveness of the three techniques in constraining the gradient scales of these variables, thus ensuring stable gradients for optimizing the physics parameters and actions. To further examine how the processed gradients will affect the actual optimization process, we experiment to compare their performances in the system identification task in Section VI.

Rugged loss landscapes/fluctuating gradients: To understand the distributions of the EMD and HMD losses, we visualize the loss landscapes of both losses over the space of the physics parameters and the skill parameters with a grid search method. For visualization, the values of two of the physics parameters are fixed to calculate the losses while varying the values of the other two parameters over their ranges with 50 even steps. The system identification losses are calculated using the optimization motion and data for soil material. For the skill parameters, the values of four of the parameters are fixed, based on the demonstration skill, and the losses are calculated, while the value of the singled-out parameter is varied over its range with 100 even intervals. The task losses are calculated using the first soil-digging task. For all loss landscapes, their approximate gradients are also calculated using the second-order central differences at the inner points and the one-sided (forward or backward) difference at the boundaries. Note that, as we are interested in the ruggedness of the losses and the directions of their gradients, these losses are centralized, and the gradients are normalized to facilitate more intuitive analyses.

Fig. 5 visualizes these losses and gradients over Young's Modulus and material density and the first two skill parameters ($\theta_{displace}$ and θ_{rotate}). They are computed with the point sets (simulated particles or real-world surface point clouds) being sampled with resolutions 20, 40 and 60, which correspond to 400, 1600 and 3600 points. Due to the page limit, the losses and gradients over Poisson's ratio and sand friction angle and the last three skill parameters ($\theta_{insert_distance}$, θ_{push_angle} , and $\theta_{push_distance}$) can be found in Section S.IV in the Supplementary Materials.

First of all, by comparing the loss distributions between EMD and HMD losses, one can see that, although they exhibit similar distributional tendencies, the HMD losses tend to be smoother. This is shown in both tasks. In Fig. 5(a), within either the negative (red) or positive (blue) parts of the losses [first row in Fig. 5(a)],

the HMD loss produces more gradients that are closer to zeros [the second and third rows of the right column in Fig. 5(a)]. In Fig. 5(b), it is obvious that the light blue curves are smoother than the light red curves. This suggests that when approaching local minima, the HMD loss is more likely to settle down with its gradients approaching zeros. Secondly, the sampling resolution seems to have a mild influence on the level of gradient noisiness. Specifically, Fig. 5(a) (the second and third rows on the right) shows that the HMD loss seems to be more rugged with increased resolution in system identification tasks, and Fig. 5(b) (red curves) shows that the EMD loss seems to be more rugged with resolution 20 and 60.

In general, with the gradients fluctuating frequently around zero when approaching local minima, gradient-descent algorithms will have difficulty converging to a solution. In addition, as the different parameters will have different gradient magnitudes, a one-size-fits-all stepsize (or stepsize schedule) is unlikely to be found. These observations suggest the need for a more adaptive stepsize selection method, such as the line search method described in Section III-F.

The next section compares the performances of different gradient operations and the line search method in the system identification task for both soil and sand materials. As the sampling resolution has a mild effect on the level of loss ruggedness, we compute the losses in the rest of the experiments with a resolution of 40 (1600 points or 40×40 height maps).

VI. RESULTS—SYSTEM IDENTIFICATION

This section presents the system identification task performance with differentiable physics and the ablation study that helps to determine the effectiveness of the EMD and HMD losses, the gradient operations (clipping, dynamic-scaling, and normalization), and the line search technique (five random seeds). To recall, we follow [5] to conduct the DPSI task by optimizing four physics parameters (Young's modulus, Poisson's ratio, material density, and sand friction angle) with gradients from either the EMD or HMD loss between the simulated and real manipulated material deformed by the same end-effector motion. In this work, we perform DPSI with soil and sand. As performing optimization with a good initial solution accelerates the convergence of DPSI suggested by [5], we also present the performance of starting from initial parameter values based on [38] (denoted by “-ManInit”).

The results are presented in Fig. 6. First of all, it can be seen that the optimization diverges with both materials and loss functions (the red and pink lines) while the use of line search greatly stabilizes gradient descent and converges to solutions with much reduced variances. This evidence demonstrates that our hypothesis of the optimization being affected by a rugged loss landscape is accurate and the line search method is effective in addressing this problem. Secondly, with a good initial solution (dark and light green curves), DPSI can converge faster to better solutions with lower variances in both materials, aligned with [5]. Thirdly, the normalization (dark and light blue curves) and dynamic-scaling (dark and light purple curves) operations perform worse than simply clipping the gradients

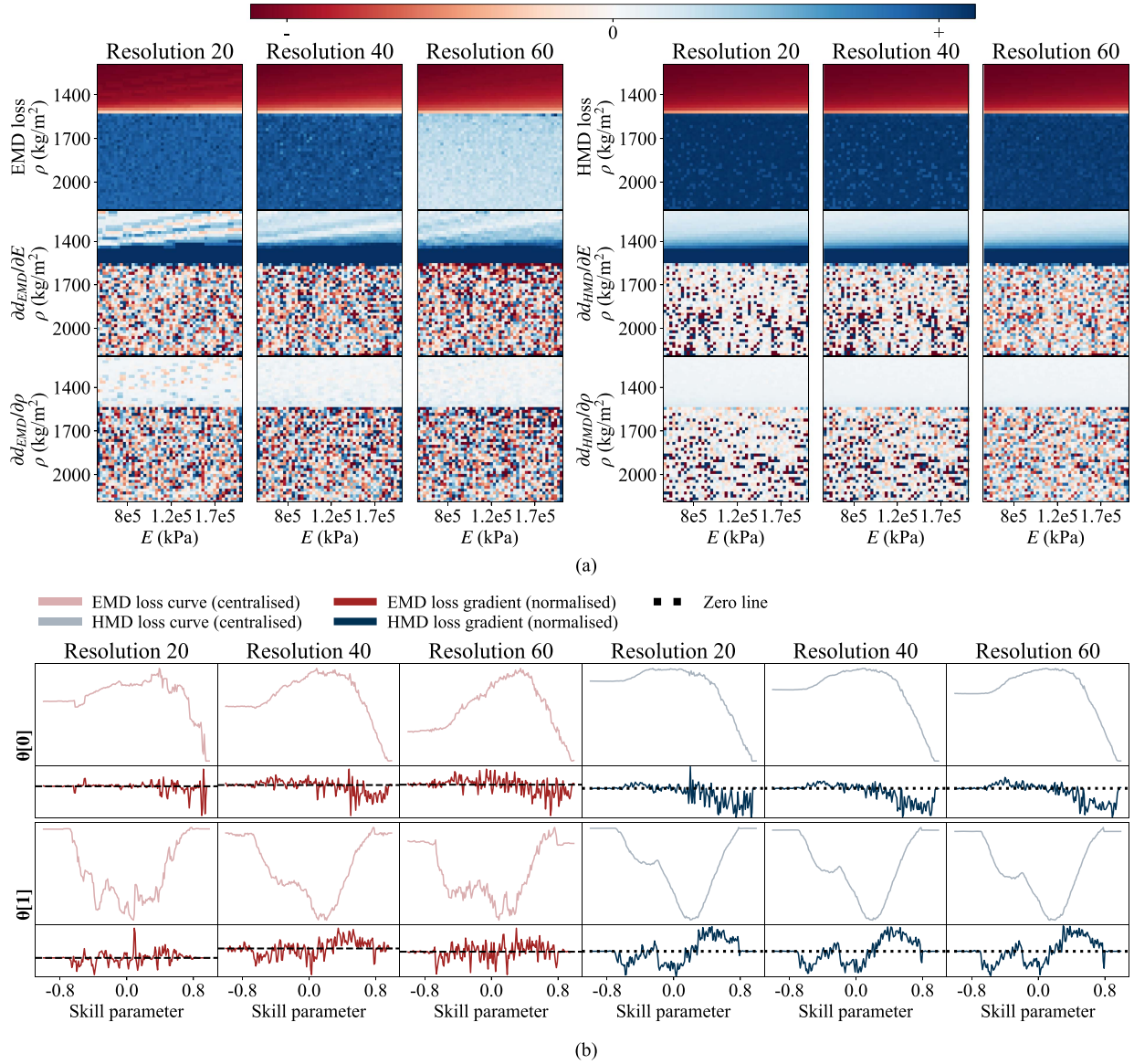


Fig. 5. Loss landscapes and their gradient distributions. (a) System identification task loss landscapes. On Young's Modulus and Material density. (b) Skill optimization task loss landscapes. On skill parameter 0 and 1.

(dark and light green curves). Lastly, the HMD loss performs slightly better than the EMD loss as all the light-coloured curves/scatter-points show slightly improved values than the dark-coloured ones (specifically, lower validation losses and variances).

In short, from the system identification results, we can be confident that the combination of HMD loss, gradient clipping, line search, and a good initial guess can more stably and efficiently lead to good optimization results.

For both soil and sand, Table III presents the physics parameter values associated with the best validation loss across five random seeds ("ClipGrad-LS-HMD-ManInit", light green lines in Fig. 6). The validation trajectories for soil and sand simulated with these parameters are presented in Fig. 7. These parameters are used in the following skill or trajectory optimizations and the RL training environment.

TABLE III
PHYSICS PARAMETERS ASSOCIATED WITH THE BEST VALIDATION LOSS OPTIMIZED USING GRADIENT CLIPPING (CLIPGRAD), LINE SEARCH (LS), HEIGHT MAP DISTANCE (HMD) LOSS, AND MANUALLY INITIALIZED SOLUTION (MANINIT)

Material	E (kPa)	ν	ρ (kg/m ³)	ϕ (°)
Soil	182683	0.242	1566	18.882
Sand	121378	0.198	1974	19.019

VII. RESULTS—DIGGING TASKS

This section examines the performance of the 6 digging tasks (Fig. 4) achieved by the proposed gradient-based skill optimization method. In Fig. 8, we plot their validation losses. The best-performing skills found by the gradient-based skill optimization

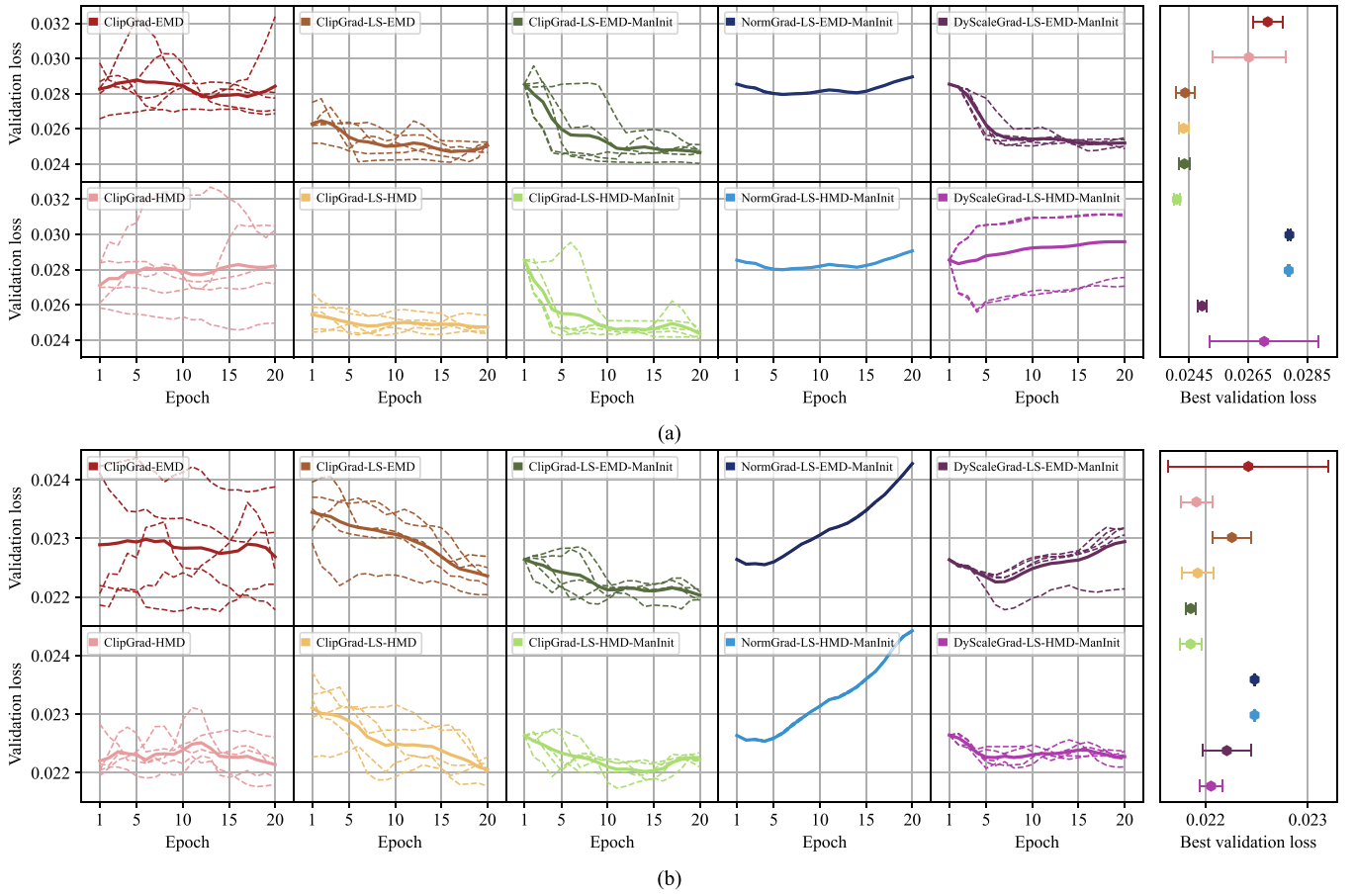


Fig. 6. System identification performances with soil/sand. LS: line search. Validation loss: the sum of the EMD and HMD loss averaged over the sampling resolution (40×40). EMD: earth mover's distance. HMD: height map distance. ManInit: manually selected initial solution. NormGrad: gradient normalization. DyScaleGrad: gradient dynamics-scaling. The validation loss values are computed by summing the EMD and HMD losses averaged by their sampling resolution (1600 points/pixels). For each material, we present the loss curves over gradient updates and the mean/std of the best validation loss values found across five random seeds. (a) Soil. (b) Sand.

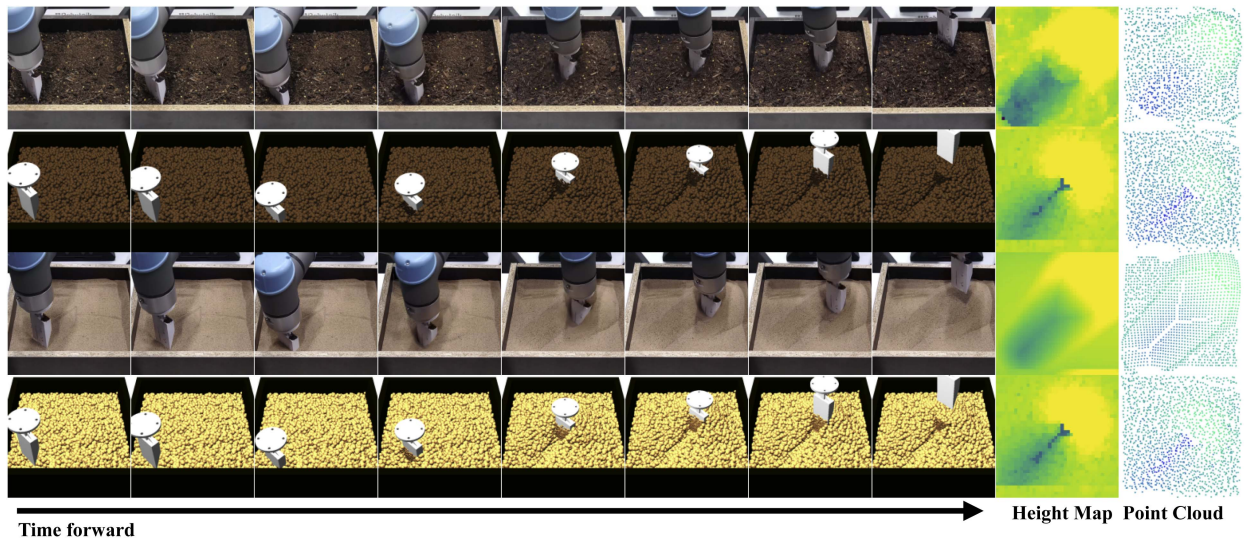


Fig. 7. Visualizing the real and simulated system identification validation trajectories with the best parameters found ("ClipGrad-LS-HM-ManInit" in Table III).

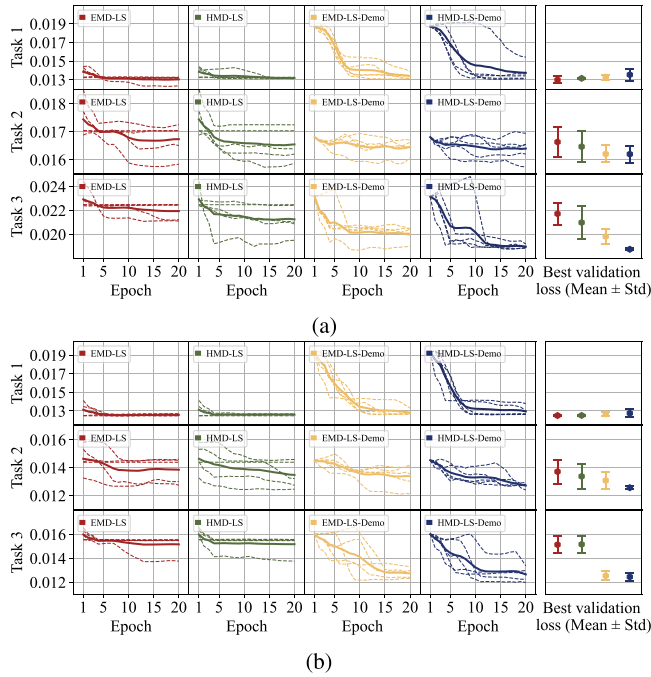


Fig. 8. Validation losses of the soil and sand digging tasks for the proposed gradient-based skill optimization method. Validation loss: the sum of the EMD and HMD loss averaged over the sampling resolution (40×40). SO: gradient-based skill optimization. EMD: Earth mover's distance. HMD: height map distance. LS: line search. Demo: demonstration-based skill prior. (a) Soil. (b) Sand.

method are deployed on the real robot. Their simulation and real trajectories, resultant point clouds and height maps, are visualized in Fig. 9.

From Fig. 8, it can be seen that the proposed gradient-based skill optimization method can successfully reduce the loss values and converge to local minima. However, the use of demonstration (skill prior) significantly accelerates optimization and leads to better performances, especially in tasks 2 and 3 for both soil and sand materials [see the yellow and blue curves in Fig. 8(a) and (b)]. For the first soil and sand digging tasks, the optimization without demonstrations performs better. This indicates that the demonstrated skill is close to a good solution, but not in every case (in the first rows of Fig. 8(a) and (b), the yellow and blue validation losses at the early epochs are significantly higher despite the demonstrated skill).

From the visualization of the simulation and real digging trajectories with the best skill parameters in Fig. 9, it can be seen that the similarity between the simulated and real dynamics is significant. The solutions can dig holes highly similar to the target shapes that were manually created. To statistically evaluate the performance, we calculate the centres, depths and areas of the real-world dug holes using their height maps. Table IV compares these statistics of the task targets and the holes dug by the best skill found by our method. It was found that, over the six tasks, the average difference in hole centres is no more than 6 mm, the average difference in hole depth is 2.5 mm, and the average area difference is 10.8 cm^2 . This proves the feasibility of deploying the optimized skills directly on a real robot.

TABLE IV
STATISTICS OF THE REAL-WORLD DUG HOLES (CM).

Features (cm)			Task 1	Task 2	Task 3
Center (x, y)	Soil	Targets	(6.79, -0.08)	(-6.23, 0.03)	(3.12, -0.97)
		Results	(5.61, -0.52)	(-6.73, -0.65)	(3.27, -0.64)
	Sand	Targets	(6.60, -0.15)	(-2.58, 0.17)	(8.57, 0.18)
		Results	(5.92, -0.63)	(-4.11, -0.51)	(8.85, -0.86)
Depth	Soil	Targets	1.74	1.48	2.43
		Results	1.69	1.48	2.78
	Sand	Targets	0.74	1.75	1.48
		Results	1.18	1.38	1.76
Area	Soil	Targets	24.84	27.72	46.80
		Results	24.12	32.04	78.48
	Sand	Targets	17.28	52.20	50.40
		Results	30.96	52.20	46.08

The averaged differences of hole centre, depth and area across the six tasks are (0.60, 0.59) cm, 0.25 cm and 10.8 cm^2 .

VIII. RESULTS—BASELINE AND ABLATION

This section presents the baseline performances and ablation study on the soil digging tasks with demonstrations. All simulation environments are configured exactly with the same initial conditions and system parameters for skill/trajectory optimization and RL training.

A. Baselines

From Fig. 10(a), it can be seen that across three soil digging tasks given demonstrations, gradient-based skill optimization (red) and CMA-MAE (yellow) consistently outperform direct trajectory optimization (green) and SAC (blue). The only occasion that trajectory optimization achieves lower validation loss than others is with a highly drastic and irregular trajectory, which happens to cause a lower loss value (see supplementary video). On the other hand, although the validation losses of the SAC agent show that it converges to a certain policy in all tasks, these policies are all stuck in local minima and found to produce very similar manipulation motions without accomplishing the tasks. The training statistics of SAC are given in Section S.V in the Supplementary Materials.

In terms of computation cost, running gradient updates for either skills or trajectories directly for 20 iterations takes about 18 to 20 minutes. In some cases, the algorithm converges to a good solution within 5 iterations with a good initial guess, which takes only about 5 minutes. Similarly, CMA-MAE also takes about 20 minutes to perform 20 iterations. The SAC agent also takes about 20 minutes to finish training for 200 episodes (200 datapoints and 19,000 neural network updates), yet hardly escapes from local poor minima.

The benchmark result provides a few insights to guide research and engineering practice. First, optimizing in the skill parameter space is significantly more efficient than in the trajectory space, even with demonstrations. Second, when an efficient and physically realistic simulator is available, learning a neural network policy with reinforcement learning methods was found to perform worse compared to gradient descent or MPC algorithms. This is somewhat unsurprising because a DRL method is

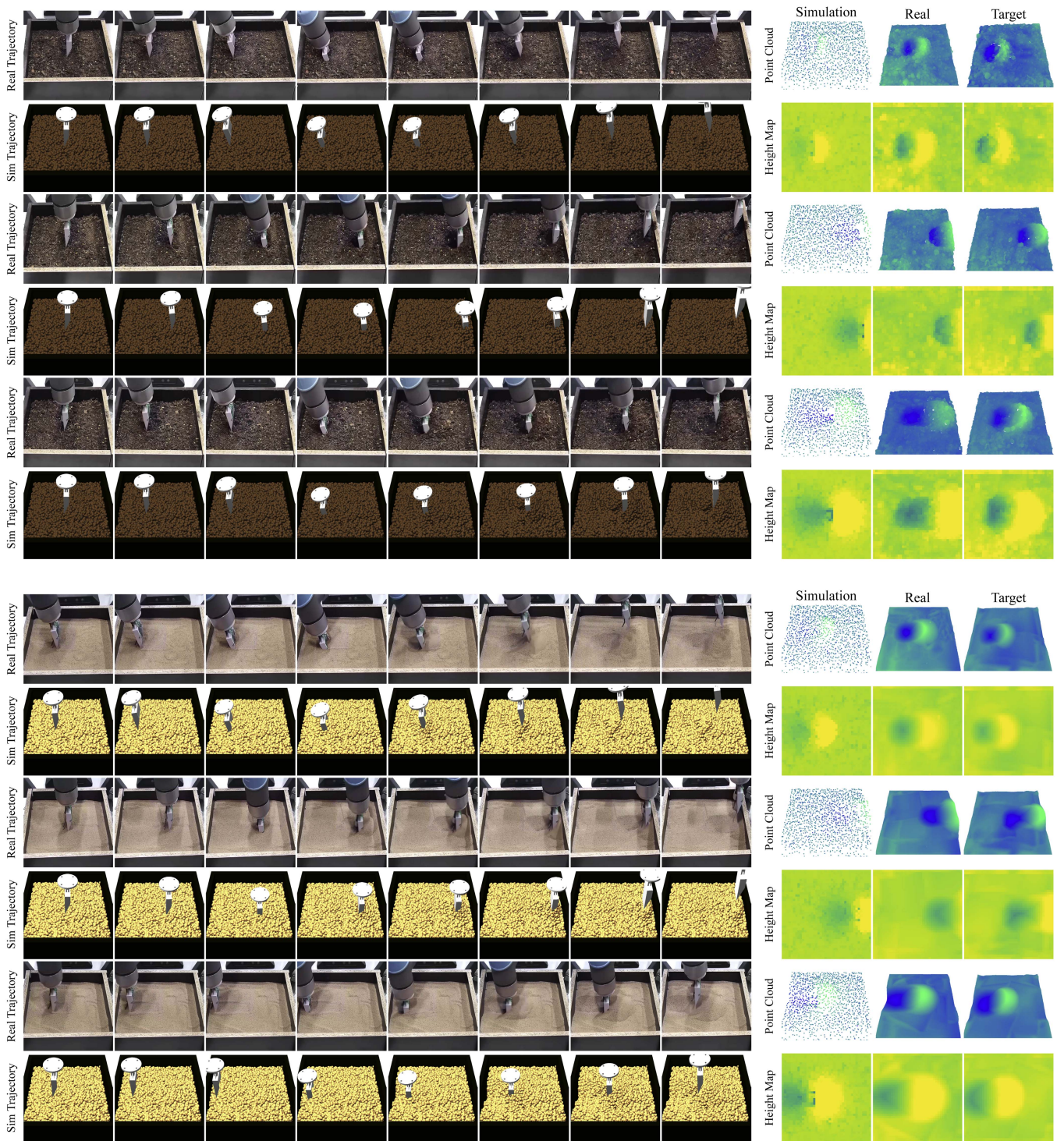


Fig. 9. Visualization of the simulated and real trajectories associated with the best validation losses for soil and sand digging tasks found by the proposed gradient-based skill optimization method.

essentially optimizing a significantly larger parameter space, in contrast to the skill parameters of size five. It suggests that physically realistic simulators, preferably differentiable, are of higher potential and importance to efficient and high-quality robotic manipulation. Lastly, our gradient descent method and the model predictive control (CMA-MAE) method perform comparably well in terms of both solution quality and computational cost

in our task setting. This is aligned with comparisons in other domains, such as image generation or classic RL tasks [41]. In practice, the hyperparameters of evolutionary algorithms, such as CMA-MAE, are generally more difficult to tune. The evolving direction of its stochastic search depends fully on the cost function, lacking consideration of the connection from the solution through the system dynamics to the post-manipulation

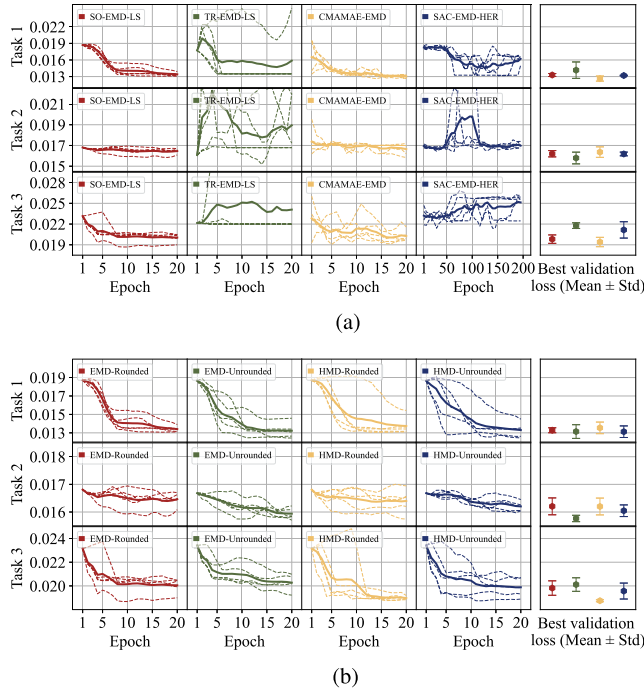


Fig. 10. Validation losses for baselines and ablation. Validation loss: the sum of the EMD and HMD loss averaged over the sampling resolution (40×40). EMD: Earth mover's distance. HMD: Height map distance. (a) Performances achieved by different methods on soil digging tasks with demonstrations. SO: Gradient-based skill optimization. LS: line search. TR: Gradient-based trajectory optimization. CMAMAE: Covariance Matrix Adaptation MAP-Annealing. SAC: Soft actor-critic. HER: Hindsight experience replay. (b) Gradient-based skill optimization (with line search) ablation on using the unrounded number of timesteps in the skill-to-action mapping.

states. This could lead to physically unreasonable solutions in complex tasks.

B. Unrounded Number of Timesteps

Fig. 10(b) compares the optimization performances with and without using an unrounded number of timesteps in the phases 1 and 4 of the skill-to-action mapping (details in Section III-B). The results show that the two cases perform similarly. In tasks 1 and 2, using an unrounded number of timesteps produces slightly better results, while in task 3, slightly worse. This result suggests that the use of an unrounded number of timesteps (a more complete gradient chain) could slightly improve the optimization performances, but not always.

IX. CONCLUSION

Granular material manipulation is of high importance to many real-world scenarios, but the field significantly lacks research and engineering experiences, particularly in small-scale and high-precision manipulation tasks with unknown material properties. This work thereby studied the challenging real-world granular material manipulation problem with unknown material properties in the context of *small-scale and high-precision* digging. A DDBot system was developed for solving such manipulation tasks using a differentiable physics-based simulator and a first-order gradient-based optimization method (RMSProp).

The differentiable simulator was tailored for granular material manipulation for fast data collection, trajectory evaluation and gradient computation, powered by GPU-accelerated parallel computing and automatic differentiation. A differentiable digging skill was designed to reduce the solution space and improve manipulation motion quality. The issues of exploding and fluctuating gradients were studied and resolved through gradient clipping and line search, enabling gradient-based optimization. To speed up convergence, a digging target-oriented demonstration method is created to serve as an initial solution or form an explorative data distribution.

The quantitative and visualization results of extensive experiments confirmed that DDBot achieves high-quality system identification and skill optimization performances with sand and soil materials. The optimized solutions were successfully deployed on a UR5e robotic arm in the real world without any adaptation (zero-shot sim-to-real transfer), achieving high digging precision (location and depth errors < 6 mm, area error ~ 10.8 cm²). In addition, with GPU acceleration and the proposed gradient treatments, DDBot achieved nearly practically applicable optimization efficiency (converges in 5 minutes with a good initial solution).

The performance of DDBot was benchmarked against gradient-based trajectory optimization, deep reinforcement learning (PointNet-backed goal-conditioned soft actor-critic) and model predictive control (covariance matrix adaptation map-annealing). Results showed that trajectory optimization and reinforcement learning (RL) failed in all tasks, while our method and model predictive control succeeded with comparable performances and efficiency. It was suggested that 1) optimization in the trajectory space for long-horizon tasks should be avoided whenever possible; and 2) when an efficient and physically realistic simulator is available, DRL methods that learn a neural network are counterproductive, gradient-free methods like CMA-MAE lack considerations of the system dynamics, whereas first-order gradient descent methods are preferred for optimization in the skill parameter space leveraging the gradients computed through the system dynamics.

Currently, the proposed system is limited to a fixed task initiation (soil/sand always flat) and optimizing only one skill. This is due to the difficulty of reconstructing 3-D scenes involving granular materials with high precision. In the future, integrating an efficient 3-D reconstruction module into the DDBot system will facilitate the optimization of multiple skills. In addition, optimizing the order of multiple skills is a discrete optimization problem. The gradient of a discrete decision-making process requires careful treatment to enable differentiability (e.g., the Softmax operation). Computing the gradient chain will become combinatorially slow as it branches for multiple skills at every skill selection stage. Studying the feasibility of differentiable simultaneous skill selection and optimization is parallel to many other research topics such as task and motion planning and hierarchical reinforcement learning. A considerable amount of research and engineering effort is needed to develop new algorithms for such tasks in the future. Finally, handling external disturbances and contaminants is another crucial and challenging direction for future research.

REFERENCES

- [1] O. Kroemer, S. Niekum, and G. Konidaris, "A review of robot learning for manipulation: Challenges, representations, and algorithms," *J. Mach. Learn. Res.*, vol. 22, no. 30, pp. 1–82, 2021.
- [2] X. Lin, Z. Huang, Y. Li, D. Held, J. B. Tenenbaum, and C. Gan, "DiffSkill: Skill abstraction from differentiable physics for deformable object manipulations with tools," in *Proc. Int. Conf. Learn. Representations*, 2022. [Online]. Available: <https://openreview.net/forum?id=Kef8cKdHWpP>
- [3] Z. Xian et al., "FluidLab: A differentiable environment for benchmarking complex fluid manipulation," in *Proc. Int. Conf. Learn. Representations*, 2023. [Online]. Available: https://openreview.net/forum?id=Cpio_BoFaE
- [4] Y. Hu et al., "A moving least squares material point method with displacement discontinuity and two-way rigid body coupling," *ACM Trans. Graph. (TOG)*, vol. 37, no. 4, pp. 1–14, 2018.
- [5] X. Yang, Z. Ji, and Y.-K. Lai, "Differentiable physics-based system identification for robotic manipulation of elastoplastic materials," *Int. J. Robot. Res.*, vol. 44, 2025, Art. no. 02783649251334661.
- [6] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
- [7] H. Yin, A. Varava, and D. Kragic, "Modeling, learning, perception, and control methods for deformable object manipulation," *Sci. Robot.*, vol. 6, no. 54, 2021, Art. no. eabd8803.
- [8] J. Xu et al., "Accelerated policy learning with parallel differentiable simulation," in *Proc. Int. Conf. Learn. Representations*, 2021. [Online]. Available: <https://openreview.net/forum?id=ZSKRQMvttc>
- [9] S. Chen, Y. Liu, S. W. Yao, J. Li, T. Fan, and J. Pan, "DiffSRL: Learning dynamical state representation for deformable object manipulation with differentiable simulation," *IEEE Robot. Automat. Lett.*, vol. 7, no. 4, pp. 9533–9540, Oct. 2022.
- [10] Y. Hu et al., "DiffTaichi: Differentiable programming for physical simulation," in *Proc. Int. Conf. Learn. Representations*, 2020. [Online]. Available: <https://openreview.net/forum?id=B1eB5xSFvr>
- [11] R. Newbury et al., "A review of differentiable simulators," *IEEE Access*, vol. 12, pp. 97581–97604, 2024.
- [12] Z. Huang et al., "PlasticineLab: A soft-body manipulation benchmark with differentiable physics," in *Proc. Int. Conf. Learn. Representations*, 2021. [Online]. Available: <https://openreview.net/forum?id=xCcdBRQEDW>
- [13] G. Klár et al., "Drucker-Prager elastoplasticity for sand animation," *ACM Trans. Graph.*, vol. 35, no. 4, pp. 1–12, 2016.
- [14] M. Wei, X. Yang, Y.-K. Lai, S. A. Taffrishi, and Z. Ji, "A physics-informed demonstration-guided learning framework for granular material manipulation," in *IEEE Trans. Neural Netw. Learn. Syst.*, 2025.
- [15] T. Tieleman and G. Hinton, "RMSProp: Divide the gradient by a running average of its recent magnitude," *COURSERA Neural Netw. Mach. Learn.*, 2012. Accessed: Apr. 20, 2018. [Online]. Available: <https://es.coursera.org/learn/neuralnetworks/lecture/YQHki/rmsprop-divide-the-gradient-by-a-running-average-of-its-recent-magnitude>
- [16] M. Görner, R. Haschke, H. Ritter, and J. Zhang, "MoveIt! Task constructor for task-level motion planning," in *Proc. Int. Conf. Robot. Automat.*, 2019, pp. 190–196.
- [17] A. Stentz, J. Bares, S. Singh, and P. Rowe, "A robotic excavator for autonomous truck loading," *Auton. Robots*, vol. 7, pp. 175–186, 1999.
- [18] D. Jud, G. Hottiger, P. Leemann, and M. Hutter, "Planning and control for autonomous excavation," *IEEE Robot. Automat. Lett.*, vol. 2, no. 4, pp. 2151–2158, Oct. 2017.
- [19] P. Egli, D. Gaschen, S. Kerscher, D. Jud, and M. Hutter, "Soil-adaptive excavation using reinforcement learning," *IEEE Robot. Automat. Lett.*, vol. 7, no. 4, pp. 9778–9785, Oct. 2022.
- [20] K. Aoshima and M. Servin, "Examining the simulation-to-reality gap of a wheel loader digging in deformable terrain," *Multibody Syst. Dyn.*, vol. 64, pp. 121–148, 2025.
- [21] V. Rondelli, B. Franceschetti, and D. Mengoli, "A review of current and historical research contributions to the development of ground autonomous vehicles for agriculture," *Sustainability*, vol. 14, no. 15, 2022, Art. no. 9221.
- [22] K. R. Aravind, P. Raja, and M. Pérez-Ruiz, "Task-based agricultural mobile robots in arable farming: A review," *Spanish J. Agricultural Res.*, vol. 15, no. 1, 2017, Art. no. e02R01.
- [23] O. Eceoglu and İ. Ünal, "Optimizing orchard planting efficiency with a GIS-integrated autonomous soil-drilling robot," *AgriEngineering*, vol. 6, no. 3, pp. 2870–2890, 2024. [Online]. Available: <https://www.mdpi.com/2624-7402/6/3/166>
- [24] Y. Wang, J. Wang, G. Zhang, M. Gao, X. Zhang, and H. Xie, "Advances in drill-based sampling technology for extraterrestrial bodies," *Space Sci. Rev.*, vol. 220, no. 6, pp. 1–58, 2024.
- [25] J. Norby et al., "Path to autonomous soil sampling and analysis by ground-based robots," *J. Environ. Manage.*, vol. 360, 2024, Art. no. 121130. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0301479724011162>
- [26] L. Dong, Z. He, C. Song, and C. Sun, "A review of mobile robot motion planning methods: From classical motion planning workflows to reinforcement learning-based architectures," *J. Syst. Eng. Electron.*, vol. 34, no. 2, pp. 439–459, 2023.
- [27] Y. Zhang, W. Yu, C. K. Liu, C. Kemp, and G. Turk, "Learning to manipulate amorphous materials," *ACM Trans. Graph.*, vol. 39, no. 6, pp. 1–11, 2020.
- [28] C. M. Mateo, J. A. Corrales, and Y. Mezouar, "A manipulation control strategy for granular materials based on a Gaussian mixture model," in *Proc. Robot 2019: 4th Iberian Robot. Conf.: Adv. Robot.*, vol. 2, 2020, pp. 171–183.
- [29] N. Tuomainen, D. Blanco-Mulero, and V. Kyrki, "Manipulation of granular materials by learning particle interactions," *IEEE Robot. Automat. Lett.*, vol. 7, no. 2, pp. 5663–5670, Apr. 2022.
- [30] S. Xue, S. Cheng, P. Kachana, and D. Xu, "Neural field dynamics model for granular object piles manipulation," in *Proc. Conf. Robot Learn.*, 2023, pp. 2821–2837.
- [31] Y. Zhu, P. Thangada, M. Ornik, and K. Hauser, "Few-shot adaptation for manipulating granular materials under domain shift," in *Proc. Robot.: Sci. Syst.*, Daegu, Republic of Korea, 2023.
- [32] J. J. Kuffner and S. M. LaValle, "RRT-connect: An efficient approach to single-query path planning," in *Proc. ICRA. Millennium Conf. IEEE Int. Conf. Robot. Automat. Symposia Proc.*, vol. 2, 2000, pp. 995–1001.
- [33] I. A. Şucan, M. Moll, and L. E. Kavraki, "The open motion planning library," *IEEE Robot. Automat. Mag.*, vol. 19, no. 4, pp. 72–82, Dec. 2012.
- [34] D. C. Drucker and W. Prager, "Soil mechanics and plastic analysis or limit design," *Quart. Appl. Math.*, vol. 10, no. 2, pp. 157–165, 1952.
- [35] J.-F. Bonnans, J. C. Gilbert, C. Lemaréchal, and C. A. Sagastizábal, *Numerical Optimization: Theoretical and Practical Aspects*. Berlin, Germany: Springer Science & Business Media, 2006.
- [36] Y. Hu et al., "Chainqueen: A real-time differentiable physical simulator for soft robotics," in *Proc. Int. Conf. Robot. Automat.*, 2019, pp. 6265–6271.
- [37] Zivid, "Zivid one plus 3D cameras," 2024. Accessed: Jan. 15, 2024. [Online]. Available: <https://www.zivid.com/zivid-one-plus>
- [38] StructX, "StructX," 2014–2024. [Online]. Available: <https://structx.com/home.html>
- [39] M. Fontaine and S. Nikolaidis, "Covariance matrix adaptation MAP-annealing," in *Proc. Genet. Evol. Computation Conf.*, New York, NY, USA: Association for Computing Machinery, 2023, pp. 456–465. [Online]. Available: <https://doi.org/10.1145/3583131.3590389>
- [40] B. Tjanaka et al., "Pyribs: A bare-bones python library for quality diversity optimization," in *Proc. Genet. Evol. Computation Conf.*, New York, NY, USA: Association for Computing Machinery, 2023, pp. 220–229. [Online]. Available: <https://doi.org/10.1145/3583131.3590374>
- [41] S. Zhao, B. Tjanaka, M. C. Fontaine, and S. Nikolaidis, "Covariance matrix adaptation MAP-annealing: Theory and experiments," *ACM Trans. Evol. Learn. Optim.*, vol. 5, no. 1, Mar. 2025, pp. 1–53. [Online]. Available: <https://doi.org/10.1145/3665336>



Xintong Yang received the bachelor's and master's degrees in mechanical and industrial engineering from the Guangdong University of Technology, Guangzhou, China, in 2016 and 2019, respectively, and the Ph.D. degree in robotics from Cardiff University, Cardiff, U.K., in 2023.

He has been a Research Associate (postdoc) with the School of Engineering, Cardiff University, since January 2023. He specialized in the robotic manipulation of real-world objects, rigid or deformable, through model-based and/or data-driven methods.

Currently, he is primarily responsible for developing a robotic platform for automatically conducting biology/chemical experiments for AI-driven science discovery. He is also working on developing a real-world-applicable robotic manipulation system.



Minglun Wei (Student Member, IEEE) received the B.Eng. degree in microelectronics from Northwestern Polytechnical University, Xi'an, China, in 2020, and the M.Sc. degree in signal processing and communications from The University of Edinburgh, U.K., in 2021.

He is currently working toward the Ph.D. degree in robotics with Cardiff University, Cardiff, U.K. Prior to starting his Ph.D. studies, he worked in industry on projects applying large language models to AI for Science. His research interests include robotic

manipulation of deformable objects and data-driven modeling of dynamical systems.



Ze Ji (Member, IEEE) received the Ph.D. degree in systems engineering from Cardiff University, Cardiff, U.K., in 2007.

He is a Reader with the School of Engineering, Cardiff University, U.K.. Prior to his current position, he was working in industry (Dyson, Lenovo, etc) on autonomous robotics. His research interests include robot manipulation, robot learning, autonomous robot navigation, physics-informed learning, computer vision, simultaneous localization and mapping (SLAM), acoustic localization, and tactile

sensing.

Dr. Ji is on the editorial boards of several journals, including IEEE/ASME TRANSACTIONS ON MECHATRONICS.



Yu-Kun Lai (Senior Member, IEEE) received the bachelor's and Ph.D. degrees in computer science from Tsinghua University, Beijing, China, in 2003 and 2008, respectively.

He is currently a Professor with the School of Computer Science & Informatics, Cardiff University, U.K. His research interests include computer graphics, geometry processing, image processing, and computer vision.

Dr. Lai is on the editorial boards of IEEE TRANSACTIONS ON VISUALIZATION AND COMPUTER GRAPHICS, *Computers & Graphics*, and *The Visual Computer*.